HTL Leonding
next level

HÖHERE TECHNISCHE
BUNDESLEHRANSTALT
LEONDING

Script Assembler – Generation of Test Scripts for KeTop Devices

DIPLOMA THESIS

submitted for the

Reife- und Diplomprüfung

at the

Department of Informatik

Submitted by:
Filip Schauer
Axel Csomány
Adam Höllerl

Supervisor:
Adrian Kern

Project Partner:
Martin Wieser, KEBA Group AG

Leonding, April 7, 2026

I hereby declare that I have composed the presented paper independently and on my own, without any other resources than the ones indicated. All thoughts taken directly or indirectly from external sources are properly denoted as such.

This paper has neither been previously submitted to another authority nor has it been published yet.

The presented paper is identical to the electronically transmitted document.

Leonding, April 7, 2026 Filip Schauer , Axel Csomány & Adam Höllerl

Abstract

This thesis presents and documents new features and improvements to the internal testing system of the *KEBA Group AG*, an Austrian technology company specializing in automation systems for industry, banking and energy applications.

After analyzing the initial problems and ways to solve them, various used technologies and concepts are explained. This includes generally known systems such as *Python* and *PostgreSQL*, as well as more specific concepts such as the *Robot Framework* and the *Monaco Editor*.

Next, the thought process and implementation of the new features and improvements is described in detail. The new functionalities include the creation of a new test script format that is based on Extended Backus–Naur form (EBNF), the definiton of a new set of keywords for the test scripts and the development of an alternative database system for the test data.

Zusammenfassung

Diese Diplomarbeit präsentiert und dokumentiert neue Funktionen und Verbesserungen am internen Testsystem der *KEBA Group AG*, einem österreichischen Technologieunternehmen, das sich auf Automatisierungssysteme für Industrie, Banken und Energieanwendungen spezialisiert.

Nach der Analyse der anfänglichen Probleme und deren möglichen Lösungen werden verschiedene verwendete Technologien und Konzepte erklärt. Dazu gehören allgemein bekannte Systeme wie *Python* und *PostgreSQL* sowie spezifischere Konzepte wie das *Robot Framework* und der *Monaco Editor*.

Anschließend wird der Denkprozess und die Implementierung der neuen Funktionen und Verbesserungen im Detail beschrieben. Zu den neuen Funktionalitäten gehören die Erstellung eines neuen EBNF-basierten Testskriptformats, die Definition von neuen Schlüsselwörtern für die Testskripte und die Entwicklung eines neuen alternativen Datenbanksystems für die Testdaten.

Acknowledgements

The team behind this diploma thesis would like to wholeheartedly thank Martin Wieser and Martin Fuchs for their continuious guidance and support throughout the making of this project.

Furthermore, we especially thank our supervisor and mentor, Adrian Kern, for his essential guidance and patience.

Filip Schauer's Acknowledgements

Without my parents and my sister by my side, I probably would have given up by now. I would like to thank them for their consistent support throughout my education, my internships, and this thesis.

Axel Csomány's Acknowledgements

I want to give huge thanks to my friends and family, that have always been supporting me.

My most special thanks is dedicated to my mother. It is thanks to her that I made it this far. May you rest in peace.

Adam Höllerl's Acknowledgements

I would like to thank my family for their constant support, encouragement and invaluable advice not only while working on this thesis, but also throughout my time in school.

Additionally, I am grateful for my friends who kept me sane, gave me a much needed escape from the demands of school and this thesis and reminded me to enjoy life beyond it.

Finally, I want to thank my thesis partners and friends, Filip and Axel, for their great teamwork and the fun we had together during this project. It was a pleasure working with you both!

Contents

Part I.

Introduction [X, F]

1. Background [X]

1.1. Testing in Industrial Systems

The testing of systems is a crucial part of the development process. It not only ensures that the system functions as intended but also helps identify flaws in areas not considered during the design phase. Testing can be performed at various stages of development, from unit testing to integration testing and system testing.

In the context of industrial automation, testing is particularly important due to the complexity and critical nature of the components involved. The systems in question often control machinery and processes that can bring about significant consequences if they fail. Therefore, a robust testing strategy is essential in order to ensure the reliability of these systems.

Companies operating in this field must develop and maintain thorough testing strategies to ensure system reliability, safety and with that the quality of their products and the satisfaction of their customers. This project focuses on improving the existing testing systems of the *KEBA Group AG*.

1.2. Company Overview: KEBA Group AG

KEBA Group AG is an Austrian technology company specializing in automation systems for industry, banking and energy applications. It designs and produces hardware and software solutions such as machine and robot controls, automated handover terminals, and electric vehicle charging stations.

For more than 50 years, they have been providing various customers with solutions to their unique problems. Among its three main branches, the Industrial Automation

branch focuses primarily on the needs of the individual situation as the hardware and its software get built.

The company was founded in 1968. Today it consists of more than 2000 employees working in 28 establishments over 16 different countries.

2. Initial Problem [X]

2.1. Current Testing Strategy and its Flaws

So called *KeTops* are used as operating devices to control machines of varying structure and complexity. These machines and their corresponding *KeTops* differ in their configuration from each customer which is why fitting test scripts are required. These test scripts examine all aspects of the device from the installed operating system to software specific behaviors.

The old *Script Assembler*, which was used in production prior to our new implementation, is able to merge a number of test scripts together for this use. The hard-coded nature and resulting limited maintainibility of this *Script Assembler* is its primary flaw. Furthermore, all data concerning tests are saved and maintained in a TestRail database, which, although functional, heavily limits the expandability and independence of the testing environment.

2.2. Acceptance Tests

The *Robot Framework* is a test automation framework designed to create and execute the test scripts in the current system. These tests are acceptance tests, which are a type of test that is created before the actual implementation of the software. The purpose of acceptance tests is to define the expected behavior of the system from the perspective of the customer or end-user. They serve as a guide to ensure that the development team is building the software according to the expectations of said customer [**?**].

Acceptance tests typically consist of terms understandable by both technical and non-technical parties, making them understandable regardless of the expertise of the reader. They intend to cover all the requirements and functionalities of the system and serve as a list of criteria for acceptance [**?**].

2.2.1. Different Types of Tests and the V-Model

The V-Model emphasizes the importance of consistent testing throughout the development process of software. In the first half of the V-Model, as seen in the lefthand side of Figure 1, the focus is on defining the concept of the system as a whole and its requirements. Further on, the definition is broken down into smaller and smaller components, until we reach the detailed design of the software. Afterwards, the implementation takes place, which is the point where the actual code is written.

The second half of the V-Model focuses on testing each corresponding unit on the same level of the first half. During each design phase, a corresponding set of tests is created, which are responsible for testing the software at that level.

For example, during the module design phase, unit tests are created, which are responsible for testing the code at its most basic level, ensuring the system functions without bugs or errors [**?**].

The V-Model displays the design and testing phases in a V shape, as seen in Figure 1.

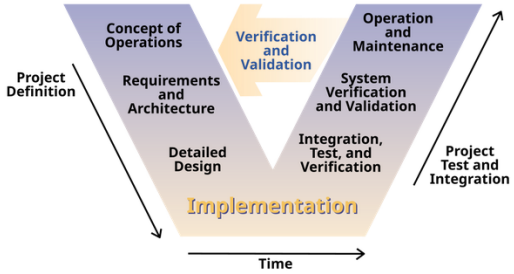


Figure 1.: V-Model of software development and testing [**?**]

2.3. Goals and Objectives

This project aims to eliminate all of the listed problems and concerns while building upon and improving the existing capabilites of the Script Assembler.

- **EBNF parsing:**
  The existing structure of the test scripts should be formed into its own EBNF language which is then integrated into the Script Assembler. This eliminates a majority of the hard-coded parts of the software. Furthermore, the editor should provide appropriate syntax highlighting when writing or editing a test script.

- **Keyword support:**

  New functionalities should be implemented, which allows the editor of test scripts to specify aspects like the version and the operating system the test has to be run on. The structure of keywords need to be precisely defined in the EBNF language.

- **Database mock:**

  The structure of the currently used *TestRail* database should be studied and an accurate mockup should be created. A seamless switch between the mock and the real *TestRail* instance has to be implemented as well.

3. Used Technologies [F]

Just like the TestRail Procedure (TRP) project for which this project aims to provide an extension for, it was mainly written in the *Python* programming language. Aside from that, other tools have also been used or are otherwise important to be mentioned. The goal of this section is to give a broad overview of some of the most important technologies that each make up an integral part of our final project.

3.1. Python

Python is a high-level, interpreted, dynamically typed object oriented programming language with a focus on allowing programmers to create applications, scripts, or connections between existing programs, quickly [**?**].

3.1.1. Modules

Importantly, Python allows for the import of *modules*, which are essentially files containing Python code which allow for importing functions, classes, or similar, into other Python files [**?**].

While it is possible for modules to be local files, they are also commonly downloaded from the internet using *package managers*. An commonly used example of a package manager for Python, which is also the only Python package manager used for this project, is known as `pip` [**?**].

The external modules used in this project include, but are not limited to:

- `lark`, which allows for parsing of a given piece of text using a custom grammar language which is based on EBNF. The core of our parser is built around this library [**?**].

- `semantic-version`, which is a small library that "provides tools to handle SemVer"[**?**]. *SemVer* stands for *Semantic Versioning*, and it refers to a scheme that intents to dictate how a software's version number is to be assigned and incremented [**?**]. This library is used to for parsing version strings for our `@VERSION` Keyword [**?**].
- `psycopg`, which allows for communication with a *PostgreSQL* database [**?**]. This library is used for our *Mockup Database* system.

3.2. Robot Framework

Robot Framework is a framework for test automation. It allows for defining so-called *Test Suites*, which are files ending in `.robot` and may contain any amount of test cases [**?**].

In the context of *KEBA Group AG*, Robot Framework is used to execute various test cases on KeTop devices. However, since our parser only acts as a preprocessor to robot script files, treating them as arbitrary text to be either included, excluded, and/or moved to different positions depending on the defined keywords and context, Robot Framework's actual syntax is, for the purposes of our project, irrelevant. Exceptions to this include the question of which characters of a standard American Standard Code for Information Interchange (ASCII) format are syntactically relevant, and its use of *significant indentation*.

3.3. Monaco Editor

The Monaco Editor is a code editor with support for features such as *IntelliSense* and *Syntax Colorization*, which also makes up the core of the commonly used *VS Code* application [**?**].

Our project makes use of its *Syntax Colorization* feature, which we instead refer to as *Syntax Highlighting* in this thesis.

3.4. PostgreSQL

PostgreSQL is an object relational database management system, it's compliant with the Atomicity, Consistency, Isolation, and Durability (ACID) properties, and uses an extended version of the *SQL* language [**?**].

PostgreSQL was our choice for our *Mockup Database* system, which is needed to allow for running our project independently from the proprietary *TestRail* test management software.

3.5. Regular Expressions

A regular expression, commonly shortened to *regex*, can be thought of small programming languages used for pattern matching in a given string. Regular expressions can be used for purposes such as checking whether or not a given string matches a certain pattern or finding patterns inside a given string. In Python, it is possible to use them through Python's `re` module [**?**].

In our project, regular expressions are mainly used for parsing purposes, as they offer an easy and compact way to find simple or complicated patterns in an arbitrary sequence of characters.

Part II.

Keywords [F, X, A]

4. Keywords Specification [F]

In programming, a *keyword* is a predefined word with a reserved meaning. Keywords can generally not be used for any other purpose, and they make up a programming language's syntax [**?**].

In our Script Assembler, we use custom keywords in a similar way, although it may be difficult to define whether or not it is useful to refer to our parser's syntax as a *programming language*.

In this chapter, the term *whitespace* is defined to refer to any amount of space or tab characters. Here, for convenience, line-breaks are chosen to specifically not fall under said definition.

4.1. Robot files

In the TRP project, the Robot Framework provides the primary scripting language that makes up the tests to be run on KeTop devices.

A Robot file contains so-called *test cases*, which are blocks commonly made up of a few lines of code [**?**]. Despite most parts of the syntax being irrelevant to the Script Assembler's workings, in order to get a feel for the nature of this project it may still be useful to have a look at an example of a Robot script file as is seen in Listing 1, which is taken from Robot Framework's official home page:

Listing 1: Example code of a Robot script file [**?**].

```
1  *** Settings ***
2  Documentation     A test suite for valid login.
3  ...
4  ...               Keywords are imported from the resource file
5  Resource          keywords.resource
6  Default Tags      positive
7
8  *** Test Cases ***
9  Login User with Password
10     Connect to Server
11     Login User             ironman    1234567890
12     Verify Valid Login     Tony Stark
```

```
13      [Teardown]    Close Server Connection
14
15  Denied Login with Wrong Password
16      [Tags]    negative
17      Connect to Server
18      Run Keyword And Expect Error    *Invalid Password    Login User    ironman
            123
19      Verify Unauthorised Access
20      [Teardown]    Close Server Connection
```

As is made clear from the example above, some lines may be preceded by whitespace. Since the Robot Framework achieves nesting by having lines be preceded by an integer multiple of exactly four spaces certain lines of code may be syntactically erroneous or interpreted differently when that amount of whitespace is changed [**?**]. This is an example of a programming convention that in this thesis will be referred to as *significant indentation*. Python, for instance, works in a similar way, requiring a predefined use of leading whitespace to achieve grouping of statements [**?**].

Despite Robot Framework's general ease of use, proficient support for extensibility, and versatile testing capabilities, the control over the specific execution flow is limited [**?**]. In the context of *KEBA Group AG*, a single *KeTop* may need to run multiple Robot script files, and a Robot script file may need to be reused for multiple KeTops. However, since it may be required for a test to be made of a single script file, it might be necessary to *stitch together*, or in other words to concatenate, the contents of multiple Robot script files into one.

Without the Script Assembler, a user may run into issues where they, for instance, would like a certain piece of code to only be included on KeTops satisfing a certain condition like being of a predefined operating system, or for certain pieces of code to be executed at a time that is either before or after the remaining parts, as the Robot Framework may not provide a useful way to achieve the required flow.

4.2. What Keywords Are

Our project allows for custom keywords, all of which being preceded by the `@` character, to be defined to act as either *options* or *modifiers*. In a provided file, these keywords must be defined at the start of the line, optionally preceded by indentation for nesting purposes. Lines containing such valid keywords will not be included in the concatenated output file.

A keyword may be defined to have either no or any number of optional or required arguments. If set, these arguments may then consist of any sequence of characters

without line-breaks that is prefixed by the keyword with at least one whitespace inbetween.

The difference between options and modifiers lies in that options do not include a *block body*, while modifiers do. A *block body* is defined to be made up of all lines of text that, when read from top to bottom, succeed the line of the keyword, up until and not including the first line that is found that does not include at least one additional level of indentation than that of the line of the keyword, or up until the end of the file, ignoring all lines that are either empty or are solely made of whitespace. The *scope* of the block body refers to the parts of the code that belong to it, as opposed to the parts of the code that do not. A level of indentation is defined to be made up of either four spaces or one tab character.

To make up for the extra level of indentation used to define the span of the body text, the parsed body text will be stripped of that one line of indentation. Non-keyword lines, and lines that do not belong to the block body of a modifier will simply be ignored by the Script Assembler and will thus be appended into the output Robot file as-is. This means that, if the provided file does not contain anything that may be matched as a keyword, the parser should leave it as-is, aside from possible minor whitespace changes. In the case of a valid Robot file, this may only not be the case in rare exceptions, as the `@` character is generally not utilized in Robot files that are to be executed on a KeTop.

4.3. Examples

4.3.1. Options

For instance, take the following code snippet, which includes the option `@NO_KETOP`, which takes no arguments:

Listing 2: Example of an Option before parsing.

```
1 @NO_KETOP
2 Install MS
```

When the line containing `@NO_KETOP` is encountered, the `no_ketop` flag, for which the concrete behaviour is explained in Chapter 5, is enabled for the output script. Since that specific line will not be included in the output script, assuming Listing 2 to be the only input to the parser, the output file may look like the following:

Listing 3: Parser result of Listing 2.

```
1  Install MS
```

4.3.2. Modifiers

Another example is using the modifier `@VERSION`:

Listing 4: Example of a Modifier before parsing.

```
1  Execute on Ketop    echo A
2  @VERSION BIOS > 1.0.1
3      IF    ${SINGLE_MODE} is $False
4          Execute on Ketop    echo B
5      END
6  Execute on Ketop    echo C
```

Here, the line two, which is made up of `@VERSION BIOS > 1.0.1`, represents a boolean condition relating to the version of the KeTop device. The block body of this condition is made up by the lines three to five. According to the specification of the `@VERSION` keyword, that block body will be included only if the condition stated in its argument is satisfied.

This means that, again assuming Listing 4 to be the only input to the parser, if the condition `BIOS > 1.0.1` is satisfied, the output may look like the following:

Listing 5: Parser result of Listing 4 if the condition is met.

```
1  Execute on Ketop    echo A
2  IF    ${SINGLE_MODE} is $False
3      Execute on Ketop    echo B
4  END
5  Execute on Ketop    echo C
```

Note the removal of one level of indentation of what used to be the block body. If, instead, the case of the condition is not satisfied, the output may instead be:

Listing 6: Parser result of Listing 4 if the condition is not met.

```
1  Execute on Ketop    echo A
2  Execute on Ketop    echo C
```

4.4. Recursion

Keywords may also be defined recursively. Recursion is generally defined as being *the process in which a function calls itself directly or indirectly* [**?**]. In the case of the Script Assembler, it refers to the fact that block bodies of modifiers may also include modifiers,

of which the block bodies may then also include modifiers, and so forth. To show this possibility, consider the following example:

Listing 7: Example of modifiers being defined recursively.

```
1  @BRACKETSTART
2      @OS WIN10
3          @VERSION BIOS >= 1.7.0
4              Upload Script To KeTop    E2Speed.ps1
```

Here, the block body of `@BRACKETSTART` is made up of lines two to four, of which the block body of `@OS` is made up of lines three and four, of which the block body of `@VERSION` is made up of line four. Despite the order of *which blocks are nested inside which* not making a difference with how all existing modifiers are specified, in general, the parser will interpret this from the outside to the innermost level of nesting, similar to how nesting is handled by most ordinary programming languages.

This means that, if the above snippet is the only input file that contains the `@BRACKETSTART` keyword, given how that keyword is specified, the part of the output script that is the so-called *bracket start* will be:

Listing 8: Bracket-start parser result for Listing 7 for satisfied conditions.

```
1  Upload Script To KeTop    E2Speed.ps1
```

In the case of both of the conditions from the respective specifications of the `@OS` and `@VERSION` keywords and with each of their respective arguments being met. Otherwise, the *bracket start* of the output file will be empty.

4.5. Concatenation and Output

In general, the files get concatenated in the same order that they are given, unless keywords that change that predefined flow are used. Then, to assure the final output Robot file is valid and executable on a KeTop, the following template code will be appended before the output of the entire parsing and concatenation operations:

Listing 9: Output file template appended before inserted test cases.

```
1  # Template file for HMI Automation
2  *** Settings ***
3  Resource          ../../../Robot/Resources/keywords.resource
4  Resource          ../../../Robot/Resources/variables.resource
5  Suite Setup       KeTop Suite Setup
6  Suite Teardown    KeTop Suite Teardown
7  Test Setup        KeTop Test Setup
8  Test Teardown     KeTop Test Teardown
9
10 *** Test Cases ***
11 # Here the test cases are inserted
```

Finally, the resulting code will be written to a file which will then be further handled to be run either locally or on a KeTop.

5. The Different Keywords [X]

This section aims to provide a comprehensive list of keywords and their corresponding definitions and uses in the context of the Script Assembler.

5.1. Options

During the making of the project, one option keyword has been implemented:

5.1.1. Option: NO_KETOP

The NO_KETOP option keyword can be used by typing the following line anywhere within the script:

```
@NO_KETOP
```

If this option keyword is present, the *no_ketop* flag will be set for the entire script. This ensures that certain components that check if a KeTop device is connected are disabled, allowing the script to run without the prerequisite of a KeTop device. This is most useful for specific test scripts that have to be run without a KeTop, like installation scripts.

5.2. Modifiers

During the making of the project, the following modifier keywords have been implemented:

- VERSION
- OS

- BRACKETSTART and BRACKETEND
- PREAMBLE

5.2.1. Modifier: VERSION

The VERSION modifier keyword can be used following the syntax:

```
@VERSION <component> <comparison> <version>
    <body>
```

Where:

- **<component>** is the label of the component that is being checked, for example "BIOS".
- **<comparison>** is a comparison operator, such as ">", "<", ">=", "<=", "==" or "!=".
- **<version>** is the version of the component that the test script is being run on, in the format of "X.Y.Z" where X, Y and Z are integers.

If the version modifier keyword is present, the body of the modifier will only be included in the final script if the condition concerning the component version is met. This allows for the creation of test scripts that behave differently based on the component version of the device.

The format of the version is intended to be functional for the rules of *Semantic Versioning*, where the three integers represent the major, minor and patch version of the component respectively:

- The major version is incremented when there are incompatible Application Programming Interface (API) changes.
- The minor version is incremented when functionality is added in a backwards-compatible manner.
- The patch version is incremented when there are backwards-compatible bug fixes.

The comparison operators regarding the version numbers examine the recency of the version in question, meaning that a version 1.2.5 would be considered higher and therefore more recent than 1.2.2. Major version changes are considered more significant

than minor version changes, which in turn are considered more significant than patch version changes [**?**].

5.2.2. Modifier: OS

The OS modifier keyword can be used following the syntax:

```
@OS <os>
    <body>
```

Where:

- **<os>** is the label of the operating system that is being checked, for example *WIN10*.

If the OS modifier keyword is present, the body of the modifier will only be included in the final script if the device's operating system matches the specified system.

5.2.3. Modifier: BRACKETSTART and BRACKETEND

The BRACKETSTART and BRACKETEND modifier keywords do not require any parameters, only the keyword itself and its body:

```
@BRACKETSTART
    <body>


@BRACKETEND
    <body>
```

If a script containing a BRACKETSTART or BRACKETEND modifier keyword is being run together with other test scripts, the body of these modifiers will be run before and after the content of all other test scripts respectively. If a batch of test scripts contains multiple scripts containing these modifiers, the bodies of the modifiers will be run in the order they appear in the batch, meaning that the body of the first BRACKETSTART modifier will be run before the body of the second BRACKETSTART modifier. The same principle applies to the BRACKETEND modifiers.

Therefore, these modifiers can be used to set up tests that require a certain amount of time to have passed, before the result can be observed. For example, if a test script

is meant to check the behavior of a device after it has been active for 30 minutes, the BRACKETSTART modifier can be used to set up the test, and the BRACKETEND modifier can be used to check the results after the 30 minutes have passed. In the meantime, other test scripts can be run between the two "bracket tests", allowing for a more efficient use of time when running a batch of test scripts.

5.2.4. Modifier: PREAMBLE

The PREAMBLE modifier keyword can be used following the syntax:

```
@PREAMBLE <index>
    <body>
```

Where:

- **<index>** is an integer that specifies the order in which the preambles are run, with 1 being the first preamble to be run.

When one or multiple test scripts containing a PREAMBLE modifier keyword is being run together with other test scripts, the body of the PREAMBLE keyword will be run before the content of all other test scripts, including lines in the body of a BRACKETSTART modifier. In the case of multiple PREAMBLE modifiers being present in the batch of test scripts, the bodies will be run in the order of their specified index.

The PREAMBLE modifier can be used to write scripts that test certain installation processes and their successful completion before the rest of the test scripts are run.

5.3. Visual Example

Consider listing 10:

Listing 10: Simple example depicting how test scripts with certain keywords would be ordered

```
1
2  Script 1:
3      @PREAMBLE 2
4      # Body of script 1
5
6  Script 2:
7      @PREAMBLE 8
8      # Body of script 2
9
```

```
Script 3:
    @BRACKETSTART
    # Body of script 3

Script 4:
    # Body of script 4

Script 5:
    # Body of script 5

Script 6:
    @BRACKETEND
    # Body of script 6
```

Here, it can be seen how some specific test scripts are arranged in the final script.

1. The bodies of the PREAMBLE modifiers are ordered first if present. In the case of multiple PREAMBLE scripts being present, the specified index is used to order them accordingly. As seen in the diagram, leaving out certain indices doesn't lead to any issues, the ordering occurs as intended.

2. The bodies of the BRACKETSTART modifiers are ordered next if present. In the case of multiple BRACKETSTART scripts being present, the order they appear in the batch is maintained.

3. At this point, the content of all test scripts not containing any order-modifying keywords are placed. Their order is determined by the order they appear in the batch.

4. Lastly, the bodies of the BRACKETEND modifiers are ordered if present. In the case of multiple BRACKETEND scripts being present, they follow the same principle as the BRACKETSTART modifiers, meaning that the order they appear in the batch is maintained.

Part III.

EBNF [A, F]

6. Overview of EBNF [A]

6.1. Motivation for Formal Syntax Descriptions

To be able to run tests automated on every kind of KeTop, there is a need to modify test scripts depending on the given environment. That is why an extended Robot Framework language was needed. The Extended Backus-Naur Form (EBNF) was chosen for this, to make this new language easily extendable and deterministic to implement if adopted by other developers.

In practice, in the field of computer science, it is clear that a precise description of a language is the most important foundation. It is therefore essential that applications, communication protocols or programs function according to clear syntactic rules so that they are always interpreted in the same way by humans and machines. Natural language provides many different ways of describing something. There are ambiguities in language, and these can lead to errors in interpretation. These findings are not new. They have been known for a long time. Nevertheless, there is still no common standard that everyone adheres to, especially when languages have been implemented independently by different developers [**?**, p. V].

Syntactic metalanguages exist to solve this problem. The goal of a metalanguage is to describe another language. This means there is a difference between the language that is described and the language used to describe it.

In programming, a metalanguage is a language used to describe the syntax and structure of another language, called the object language. For example, BNF and EBNF are metalanguages used to define programming languages such as ALGOL. The structure of sentences and expressions in a metalanguage can be defined using a so-called metasyntax. For example, one could indicate that the word “noun” can itself function as a noun in a sentence by writing: “noun” is a `<noun>` [**?**].

6.2. The Backus-Naur Form (BNF)

One of the most important metalanguages for programming has been BNF (the so-called Backus-Naur Form). BNF started with ALGOL 60 in 1960. ALGOL 60 needed a clear, formal way to describe its syntax. John Backus and Peter Naur therefore created BNF. It made it possible to define language rules precisely. This was the first time syntax was described formally. The history of ALGOL 60 is crucial because it gave BNF its first real use. BNF became the foundation for defining many programming languages. It is still essential in compilers and language design today [**?**, p. 3].

As BNF was first used for ALGOL 60 and became the model for many later languages, over time, people added changes or extensions, which created some confusion. Despite this, BNF showed the importance of clear, formal syntax.

Therefore, Extended BNF was later developed from BNF and included the most common extensions. The large amount of variants motivated the standardization effort that resulted in the standard named *ISO/IEC 14977* by the International Organization for Standardization (ISO) [**?**, p. V].

6.3. The Evolution to Extended BNF (EBNF)

For early programming languages such as ALGOL 60, BNF was adequate. However, the increased complexity of modern languages required a more systematic notation to describe their syntax effectively.

Therefore, the so-called Extended Backus-Naur Form (EBNF) was introduced. The inventor of Extended BNF - EBNF, is Niklaus Wirth, a Swiss computer scientist. EBNF is based on BNF but it adds more elements.

> “One cannot fail to notice the lack of "common denominators."” [**?**]

Niklaus Wirth stated this in his first article in the journal “Communications of the ACM” in 1977 about EBNF.

He explained that there is a lack of common standards or unified fundamental principles across different language definitions and notational forms. Each language definition tends to use slightly different variants of the Backus-Naur form based notation. There is no generally accepted standard, and the differences are often small but unnecessarily

varied. He stated that a more uniform notation would improve scientific work and comparison. EBNF is his solution for this problem.

As mentioned EBNF is clearly based on BNF, but it adds more elements. EBNF can define exact numbers of elements, describe exceptions, add comments, use flexible rule names, and allow extensions. This makes grammar definitions shorter and it is easier to read them. But EBNF also has restrictions. It can only handle languages with elements in order and is not able to describe very complex grammar [**?**, pp. VII-VIII].

The main disadvantage of BNF according to ISO/IEC 14977 was:

> "Backus-Naur Form (used in ALGOL 60) has problems if the metasymbols < > ::= occur in the language being defined. Some common forms of construction (e.g. comments) cannot be expressed naturally; other constructions (e.g. repetition) are long winded." [**?**, p. VII]

The added advantages, but also the limitations of EBNF, will be discussed in more detail in later sections and chapters.

6.4. Wirth's Approach to Syntax Definition

Having established the historical need for a unified standard in the previous section, it is now necessary to examine the concrete syntactic rules proposed by Niklaus Wirth in 1977. In his article published in the journal Communications of the ACM (CACM), Wirth addressed the existing problems of syntactic notation directly. His contribution marked an important step in the formal development of language definitions.

Wirth did not intend to introduce a completely new theoretical concept. Instead, his objective was to improve the existing Backus-Naur Form in a systematic way. BNF had already proven to be useful. However, it contained ambiguities and practical limitations.

The notation presented in his article was therefore designed to be more practical and clearer in structure. It focused on improving readability and standardization rather than expanding theoretical expressive power. The goal was not innovation for its own sake. The goal was clarity, consistency, and easier application in real technical environments.

Wirth listed the following properties to extend BNF [**?**]:

1. The notation clearly separates meta symbols, terminal symbols, and nonterminal symbols.

   Example: `syntax = "production".`

   Here, `syntax` is a nonterminal symbol. `"production"` is a terminal symbol. The equals sign and the dot are structural meta symbols.

2. Symbols used for structure can also be used as normal language symbols. This avoids artificial restrictions.

   Example: If quotation marks are used as meta symbols, they can still appear inside text if written twice, like `""` in many programming languages.

3. The notation contains a simple way to express repetition. This reduces the need for recursion in simple cases.

4. The notation avoids explicit representation of the empty string, such as by using `<empty>` or ε.

5. The notation is based on the ASCII character set. This makes it easy to use in most technical environments.

In his article, he showed the power of EBNF by defining its own syntax as an example (see Figure 2):

> "This meta language can therefore conveniently be used to define its own syntax, which may serve here as an example of its use. The word identifier is used to denote nonterminal symbol, and literal stands for terminal symbol. For brevity, identifier and character are not defined in further detail."[?]

```
syntax     = {production}.
production = identifier "=" expression ".".
expression = term {"|" term}.
term       = factor {factor}.
factor     = identifier | literal | "(" expression ")" |
             "[" expression "]" | "{" expression "}".
literal    = "'" character {character} "'".
```

Figure 2.: Definition of EBNF in EBNF in "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" from N. Wirth [?]

6.5. Core Syntactic Extensions

The following overview focuses in more detail on the differences between BNF and Extended Backus-Naur Form (EBNF) as described in Concepts of Programming Languages by Robert W. Sebesta [**?**, pp. 149-151].

It is worth noting that these extensions do not add to the descriptive capabilities of BNF. Their purpose is simply to make formal grammars easier to read and write. BNF is theoretically sufficient for describing context-free languages, but EBNF provides a more concise notation for complex structures.

6.5.1. Core Syntactic Extensions in EBNF in detail

Sebesta identifies three primary extensions common to most versions of EBNF:

- **Optionality** (`[...]`):
  Brackets are used to denote a part of a right-hand side (RHS) that may occur once or not at all.
  Example: `[meow]` means meow or empty.
- **Repetition** (`{...}`):
  Braces indicate that the enclosed part can be repeated any number of times, including zero. This replaces the need for recursion when describing simple lists.
  Example: `{chirp}` can be empty, chirp, chirpchirp, chirpchirpchirp, and so on.
  Example for the usage for lists: `list = <item> {<item>}`, which represents a list of one or more items.
- **Multiple-Choice Options** (`(...|...)`):
  A set of alternatives can be written in parentheses, with the vertical bar (i.e., the OR operator `|`) used to separate the possible choices.
  Example: `(pancake | waffle)` represents either pancake or waffle.

In this notation, brackets, braces, and parentheses serve as metasymbols rather than terminal symbols of the language being described [**?**, pp. 149-151].

6.5.2. Further Syntactic Extensions in EBNF based on ISO/IEC 14977

However, there are more differences between BNF and EBNF in the ISO/IEC 14977 definition: [**?**, p. VII]

- **Defining an explicit number of items:** In some cases it is needed to describe, that e.g., a field must have an exact length. In BNF, this is difficult. Often, the programmer has explained it then informally in English. Now EBNF solves this by allowing an integer followed by a repetition-symbol (`*`). With this solution, the programmer can specify the exact number of items. This makes rules much shorter.
- **Exceptions:** Sometimes it would be easier to define a rule by specifying what is not allowed. But BNF cannot easily say "any character except a quote". EBNF has a solution for that. EBNF uses an except-symbol (`-`) to define rules by exclusion. This makes definitions shorter and easier. Therefore, for example, a programmer can define a group of symbols and then subtract specific exceptional cases, such as "any character except a quote".
- **Formal comments:** Comments are an important feature, that BNF is not offering. EBNF provides a formal way to add comments using `(*` and `*)`. And these notes do not change the machine's logic.
- **Flexible names:** In BNF, names for categories often need brackets. EBNF uses an explicit comma (`,`) as a so-called concatenate symbol to link parts of a rule. This removes the restriction that category names or meta-identifiers must be single words or enclosed in brackets, allowing for more flexible naming. It also ensures that the layout of the syntax like spaces or line breaks has no effect on the defined language.
- **Extensions:** Programmers can add their own features to EBNF. These are called "special sequences" and are placed between question marks (`?`). This makes the start and end of any extension very easy to find. Extensions were used several times in earlier versions of the solution presented in this thesis, but were eliminated in the final version because there was no longer a need for them.

6.5.3. Comparative Notation BNF and EBNF

While classical BNF is theoretically sufficient for describing any context-free language, it is often complicated and repetitive in practice. The main cause therefore is, that it relies, as described in the previous chapter, exclusively on recursion to express lists and optional elements. In the following illustration one can see the advantage of EBNF introducing specialized meta symbols, such as braces, brackets, and parentheses [**?**, p. 153].

BNF and EBNF Versions of an Expression Grammar

BNF:

```
<expr> → <expr> + <term>
       | <expr> - <term>
       | <term>
<term> → <term> * <factor>
       | <term> / <factor>
       | <factor>
<factor> → <exp> ** <factor>
         | <exp>
<exp> → (<expr>)
      | id
```

EBNF:

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → <exp> { ** <exp>}
<exp> → (<expr>)
      | id
```

Figure 3.: Comparison between BNF and EBNF by Sebesta [**?**, p. 153]

As demonstrated in the example in Figure 3, the structural differences lead to a significant reduction in the number of rules required:

- **Repetition via Braces in this example:** In the BNF version of the expression grammar, an `<expr>` must be defined recursively to handle a sequence of additions or subtractions. EBNF replaces this recursion with braces `{...}`, which denote that the enclosed part can be repeated indefinitely or left out entirely.
- **Multiple-Choice Options in this Example:** The example uses parentheses `(...)` combined with the OR operator `|` to group choices. This for example allows the EBNF version to handle both addition and subtraction in a single line. In BNF it requires a more complex approach.

6.5.4. Modern Variations and Standardization

While an official standard exists (ISO/IEC 14977:1996), it is rarely used in practice. There have several variations of EBNF emerged in recent years, which deviate from the standardized syntax, such as using a colon instead of an equal sign or a dot instead of a semicolon. Also the project described in this thesis partially deviates from ISO standards due to using the EBNF dialect of the Lark Python library.

6.6. ISO/IEC 14977 Standards for Assembler Logic

While the previous chapters describe the advantages of EBNF in terms of readability and compactness, the ISO definition has an additional function. The ISO standard ISO/IEC 14977 also defines strict technical rules.

These are very important for error-free processing by an automated tool such as a script assembler, as they ensure that the grammar definition remains mathematically unambiguous.

6.6.1. Operator Precedence

For every EBNF grammar, the strictly defined hierarchy of meta-symbols is essential, as it prevents ambiguities in the rules. The order of the symbols is clearly defined in the ISO standard without any leeway and is specified as follows: (the strongest is the repetition symbol. This takes precedence, followed by the exclude symbol, and so on).

1. Repetition symbol (*): For specifying exact numbers of repetitions.
2. Except symbol (-): For defining sets by exclusion.
3. Concatenate symbol (,): For explicitly linking consecutive elements.
4. Definition separator symbol (|): For separating alternatives.
5. Defining symbol (=): Formally assigns an expression to a nonterminal.
6. Terminator symbol (;): To clearly mark the end of a production rule.

[**?**, p. 1]

6.6.2. Layout Neutrality Through Gap Separators

Ideally, a script assembler should be able to process scripts regardless of their visual formatting. The ISO standard is helpful here, as it defines so-called "gap separators" (such as spaces, tabs, or line breaks) [**?**, pp. 5]. These characters have no formal meaning outside of terminal strings. This has the significant advantage of allowing for flexible grammar design without affecting the assembler's logic. Since gap separators and comments have no formal effect on the defined language, the script assembler can ignore these characters during analysis, which, for example, increases its robustness against different programming styles.

In summary, several advantages arise. The grammar definition remains mathematically consistent for automated tools. Furthermore, it is beneficial that the grammar can be made more readable and understandable for human readers through formatting and annotations.

6.6.3. Syntactic Exceptions

As mentioned earlier, the Except symbol (-) allows the assembler to define specific instruction sets. An example of this would be "all characters except a semicolon." However, the ISO standard also imposes very clear technical limitations on this function, as otherwise paradoxes could arise:

> "If a syntactic exception is permitted to be an arbitrary syntactic factor, Extended BNF could define a wider class of languages than the context-free grammars, including attempts which lead to Russell-like paradoxes... Such a license is undesirable and the form of a syntactic exception is therefore restricted to cases that can be proved to be safe." [**?**, p. 2]

A good explanation for the Russell Paradox, formulated by Bertrand Russell, is the so-called barber's example: In a village, the barber shaves precisely all the men who don't shave themselves, and only these men. The question of whether the barber shaves himself leads directly to a contradiction: If he shaves himself, he shouldn't, by definition; if he doesn't shave himself, he should [**?**].

This example illustrates that such self-referential definitions can lead to logical inconsistencies in formal systems.

6.6.4. Alternative Representations for System Compatibility

To guarantee that a grammar functions on different systems, the ISO standard allows the substitution of certain special characters if they are missing. For example, the characters `(:` and `:)` can replace `{` and `}`, or a period (`.`) can replace a semicolon at the end of a rule. This ensures that the grammar remains readable and functional on different platforms without losing its original structure [**?**, p. 5].

6.6.5. Special Sequences as a Formal Interface for Assembler Extensions

Although special sequences were already described in the previous section as a general extension option, they are listed again here to specifically clarify their "technical" role as placeholders. In the logic of a script assembler, they serve as a standardized mechanism to mark parts of the language that cannot be defined by EBNF itself (e.g., specific character sets) [**?**, p. VII].

By enclosing them in question marks (`?...?`), the grammar remains formally valid for the assembler tool. In this work, they thus can provided the flexibility for possible future functional extensions without violating the fundamental syntactic rules of the ISO standard [**?**, p. 3].

6.6.6. Evaluating the Standard for our Script Assembler

After introducing the theoretical foundations of EBNF, it is necessary to evaluate how the ISO standard was applied in the context of the script assembler. The goal of the project was not to implement a complete parser for the Robot Framework. Instead, the focus was on building a preprocessor for specific control tags such as `@OS`. For this reason, the resulting grammar is not a strict implementation of the ISO standard. It represents a pragmatic combination of formal rules and practical requirements.

A central aspect of this evaluation is the concept of layout neutrality. The ISO standard defines spaces, tabs, and line breaks as so-called gap separators. These elements have no formal meaning and can be ignored by the parser. This principle improves flexibility and readability. However, it could not be applied completely in this project. Inline

spaces and tabs between control tokens are treated as insignificant and are therefore ignored. This allows test scripts to remain flexible in their formatting.

Line breaks, however, require a different treatment. Robot Framework scripts are structured line by line. This means that line breaks carry semantic meaning. They define the structure of the script. Therefore, line breaks could not be treated as layout-neutral elements. Instead, they were explicitly defined as structural tokens in the grammar. This is a necessary deviation from the ISO specification to ensure correct processing of the input.

Further deviations from the standard were made in the handling of syntactic constraints. The ISO standard introduces syntactic exceptions to define rules by exclusion. However, this mechanism can become complex and difficult to understand. In some cases, it can even lead to problematic definitions. For this reason, syntactic exceptions were not used in the assembler.

Instead, the implementation relies on regular expressions. Regular expression (regex) provides a practical way to capture and process text patterns. It allows the parser to treat sections of Robot Framework code as plain content. The internal structure of these sections is not analyzed. This simplifies the grammar and reduces its complexity. As a result, the grammar becomes easier to read and maintain.

7. Approaches to EBNF [A]

7.1. Requirements

When creating new formal grammar, one quickly is confronted with a common problem in software development. On one side, you want a clean design, but on the other there is this old legacy system at the company that everyone is used to. For the development of the Script Assembler, this means that it must not only be able to process multi-line structures correctly, but also that the legacy system used at KEBA must continue to function with the new formal grammar, and unfortunately, backward compatibility is a must-have.

The legacy system used before the Script Assembler is a simple preprocessing system, that was based on basic string matching. This solution uses markers that start with an `@` sign. Those were already existant:

- **`@BRACKETSTART, @BRACKETEND:`** These strings marked sections of code. The system just checked if a line exactly matched these strings to know if the following Robot Framework code should be appended at the beginning or the end of the final script.
- **`@NO_KETOP:`** This was a global option that meant a KeTop device was not needed for the test. It could be written anywhere in the file, and the script just searched the whole document for it without looking at the context.

This solution checked if a specific string was present inside a file or a line and did not use a formal parser. This worked fine for isolated, simple use cases, but not with complex, context-sensitive multi-line structures or dynamic block delineations. Without a parser, the hardcoded logic simply becomes an unmaintainable mess over time.

For this reason, designing the new formal grammar became hard to manage, because three major restrictions had to be taken into account, as clarified in the first meetings by KEBA:

- **Deterministic Parsing:** The system needs strict, unambiguous rules to cleanly differentiate syntactic constructs and reliably deal with multi-line scopes.
- **Backward Compatibility:** It was a strict requirement that the old legacy scripts kept working exactly like before. The functionality had to be preserved. A disruption of the established workflow for the testing team was not an acceptable option.
- **Future Extensibility:** The system must be very robust, because it was stated, that new custom annotations and hardware-specific conditions will definitely be added in the future. A complete rewrite of the parser for every new feature would simply not be an option. .

The testing team at KEBA was already deeply familiar with the `@` prefix. Early design discussions brought up the idea to introduce distinct prefixes to simplify the parsing. The suggestion was to introduce `#` for global options, and `&` for block statements. The idea was, that the tokenization process would become significantly easier and more accurate this way. The evaluation feedback then showed the problem that the distinct prefixes break all existing test suites. A complete disruption of the developers workflow would be the result.

This created a conflict. A new parser is strictly necessary, but the legacy `@` prefix and the overall backward compatibility must still remain intact. This conflict dominated the first design phase and led to the following solution. Though the strictness of the "Backward Compatibilty" restriction was not as clear at that point. To establish a good understanding of the available options and technical trade-offs, two contrasting EBNF approaches were modeled. The evaluation happened before the final syntax design was established.

7.2. Minimal EBNF Approach

One of the two strategies explored was the minimal EBNF approach. The idea behind this specific design was to capture the general script structure as broadly as possi-

ble. Theoretically, this offers maximum flexibility and makes future extensions very easy.

Listing 11 demonstrates this broad structural definition. The grammar does not strictly define every possible keyword. Instead, it simply distinguishes between standard Robot Framework lines, single-line operators, and multi-line operators.

Listing 11: The Minimal EBNF grammar

```
script = {line, nl}, [line].
line = robot | operator | multiline_operator.
robot = ? robot code line ?.
operator = '@', word, { ws, argument }.
multiline_operator = operator, [nl], '{' { robot, nl } '}'.
nl = "\n".
ws = " " | "\t".
argument = ? some argument for example comparison operators or strings ?.
```

7.2.1. Problems of the Minimal Approach

While the parser implementation stays lightweight and adaptable with this approach, there is a blind spot and it completely fails at validating the actual code structure:

1. **Unresolved Ambiguities:** The grammar failed to strictly define and categorize specific keywords. It could not differentate between standalone options (e.g., `@NO_-KETOP`) and block-defining modifiers (e.g., `@VERSION`). The parser was basically blind to the meaning of the words and no correct validation was possible.

2. **Scope Definition Conflicts:** A critical issue emerged during the following design phases when indentation was chosen over explicit brackets (e.g., `{` and `}`) as the primary method for defining scopes. The minimal grammar was just defined too loosely for it to be able to differentiate between either an option that has a body despite it not being allowed to (e.g., developer misinterpreting functionality). The parser was not able to detect the syntax error and invalid code might pass through.

7.3. Detailed EBNF Approach

While the Minimal Approach explored maximum flexibility, a completely different and rather strict grammar was developed. This EBNF approach uses clear rules and a strict separation of all constructs. The main goal was a much better parsing clarity.

As shown in Listing 12, this iteration uses distinct prefixes for different types of constructs. It also introduces explicit termination symbols to make the structure clear.

Listing 12: The Detailed EBNF grammar

```
1  S = { ' ' | '\t' } ;
2  robot_line = S , ?robot_script_line? , S ;
3  option_line = S , '#' , S , option , S ;
4  statement_line = S , '&' , S , statement , '\n' , { line , '\n' } , S , '&/' , S
       , '\n' ;
5  line = robot_line | statement_line | S ;
6  script = { option_line , '\n' } , ( ( { line , '\n' } , [line] ) | [option_line] )
       ;
7
8  statement = version ;
9  option = "NO_KETOP" ;
10 section = "BRACKETSTART" | "BRACKETEND" ;
11
12 version = "VERSION" , S , ( '>' | '<' | '=' | '>=' | '<=' ) , S , ?version_number?
       ;
```

Looking closer at the code, on can see, that in this specific design, `#` was introduced for global options, and `&` was utilized for statements, which were explicitly terminated by `&/`. Since this approach was never fully completed `@` was not used yet, but the idea was to use it only for sections. The grammar also checks the expected arguments very strictly. For instance, the `VERSION` statement forces the user to use mathematical comparison operators and a specific version number format (line 12).

7.3.1. Problems of the Detailed Approach

This approach was very precise and stable. Unfortunately, there were two severe drawbacks that made it unusable for the final product:

1. **Loss of Backward Compatibility:** The introduction of `#` and `&` caused problems with the existing `@` prefix used by the existing option. Changing all the old code was not an option. Furthermore it seemed more intuitive to just have one prefix for all types.

2. **Excessive Rigidity:** The rules for how an argument must look are too strict. For example, the forced mathematical operators in the `version` rule make the grammar very rigid. This means every time a new modifier is added with new arguments, the EBNF has to be updated to match the arguments exactly. This made the Script Assembler harder to extend.

7.4. Conclusion

Testing these two different approaches provided important details for the final EBNF. The Minimal Approach showed the possible dangers of structural ambiguity. It is clear that the parser needs to "understand" the difference between a standalone option and a block-defining modifier. On the other hand, the Detailed Approach proved that excessive rigidity is also not completely perfect. Making the parsing process too easy for the machine just makes extending unneccessary hard.

The conclusion from this evaluation phase was that the final grammar definitely needed a hybrid design to work. On the inside, it requires the precision of the Detailed Approach, which means in detail that all keywords must be strictly defined and categorized internally so the parser can recognize valid scopes and modifiers correctly. But at the same time, it still has to keep the same look and the backward compatibility of the old legacy approach by using the `@` prefix, everywhere, while also making the argument structure much more relaxed to make it easier to add extensions later on.

The idea of explicit termination symbols like `&/` was completely dropped in favor of indentation-based scoping, and the old "sections" were just changed into standard modifiers.

The combination of all these different requirements finally leads to the final functional grammar used in Script Assembler.

8. EBNF's Indentation Problem [F]

8.1. EBNF's Indentation Problem

To get to our final EBNF, we have to represent all of the following:

- The entire script is made of *lines*, seperated by line breaks, each of which can be a *robot line*, an *option line*, or a *modifier line*.
- There are two different types of keywords: *options* and *modifiers*. Valid *options* include `@NO_KETOP` and valid *modifiers* include `@VERSION`, `@OS`, `@PREAMBLE` `@BRACKETSTART`, and `@BRACKETEND`.
- It should be possible for both *modifiers* and *options* to have optional arguments until the end of the line. The nature of the argument itself or whether or not a specific keyword should require or allow for an argument should be, for simplicity and ease of extensibility, handled by the parser and therefore do not lie within the scope of the `ebnf`.
- *Modifier lines* include a *block body*, which is also made up of additional *lines* which may be a *robot line*, *option line*, or, recursively, a *modifier line*.
- A block body is marked by each line of text being preceded by an indentation character, which may either be a tab character or four spaces.
- All other lines are handled as *robot lines*.

Note that in the list above, there is a differentiation between the EBNF object *lines* and the term *line of text*, which stands for a literal string terminated by a new line character or the end of the file.

However, what follows is a non-rigorous proof by contradiction that it is impossible to fully define a language that meets all the above requirements in EBNF.

8.2. Impossibility of Indentation

What follows are three statements that, for the purposes of this non-rigorous proof, are each assumed to be true.

Firstly, it is possible to specify the *modifier line* as a single, re-usable component, because it must be possible for a *modifier line* including its *block body* to refer back to itself to allow for recursion.

Secondly, as stated in the requirements above, a *block body* of a *modifier line* may be made up of any number of lines of text, each of which should be preceded by an indentation.

Thirdly, when a *modifier line* B is defined inside the *block body* of another *modifier line* A, it is possible to specify that each *line* of the *block body* of *modifier line* B should be preceded by indentation.

However, the third statement would require *modifier line* B to be defined differently to *modifier line* A, as each of its line of text of its *block body* must be preceded by an indentation as stated in the second statement. This means that, in order to require indentation as prefix for every inner *line*, a *modifier line* may not allow for the same kind of *modifier line* as would be used in other parts of the script to be used in its *block body*, which violates the first statement which requires *modifier lines* to be defined as single and re-usable components. Therefore, it is not possible for all three statements to be true at once.

8.3. Our Solution

A simple solution to this problem is to not define the entire language in EBNF, but rather, to require a *preprocessor* to sanitize the input script in a way that makes it suitable for EBNF.

In our project, the task of the preprocessor is to transpile the indentations required for defining the scope of a block body to a more EBNF-friendly way.

Instead, the preprocessor uses indentation to determine the scope of a block body. Then, that indentation is stripped from all lines of the block body, and a *special sequence of text* is appended below the block body to specify its end.

For instance, consider the following code snippet:

Listing 13: Example of modifiers being used with indentation.

```
1 Execute on Ketop    echo A
2 @VERSION BIOS > 1.0.1
3     Execute on Ketop    echo B
4     @OS WIN10
5         Execute on Ketop    echo C
6 Execute on Ketop    echo D
```

After the preprocessor step, assuming all conditions to be met and the string `[placeholder]` to be used as the *special sequence of text*, it may be turned into the following:

Listing 14: A possible preprocessor output from listing 13.

```
1 Execute on Ketop    echo A
2 @VERSION BIOS > 1.0.1
3 Execute on Ketop    echo B
4 @OS WIN10
5 Execute on Ketop    echo C
6 [placeholder]
7 [placeholder]
8 Execute on Ketop    echo D
```

Here, the preprocessor removes a single indentation from lines 3 to 4 and two indendations from line 5. To avoid losing information on which lines define the *block body* of the *modifier*, two lines are appended after line 5, both of which are made up of solely our *special sequence of text*.

With this method, the EBNF does not have to take into account the fact that an inner block body would otherwise require every line to have a prefix. Instead, a *modifier line* is able to *sandwich* any lines in between a clear beginning (which is the line that the *modifier* is defined) and a clear end (which is the mentioned *special sequence of text*).

The choice of that *special sequence of text* should be one that is not expected to be in the input script. For the Script Assembler, we chose `@END`, as we have already reserved the `@` character for our purposes, there is no keyword with that name, and it still leaves for human-readable code if a user were to look at the code after the preprocessor but before parsing.

A side effect of this is that a user may be able to specify the scope of a block body using `@END` as opposed to how it is defined in the specification. However, this may be mitigated: If the preprocessor does not allow for the use of `@END`, i.e. if it, for instance, throws an error upon encountering the symbol, it will not be possible for the user to use or reserve this internal string. A different solution may be to allow `@END` to be used

as an alternative to the indentation style. However, as we believed this method to be prone to human error, we decided to go with the former solution instead.

9. Our EBNF [F]

From the requirements above, the following code block is the final EBNF that was settled on. In this chapter, we will go into detail about its syntax and meaning.

Listing 15: The complete final EBNF that we developed.

```
start = { N } , { line , { N } } ;

robot line = ? regex: /[^(@|\n)].+?(?=\n)/ ? ;

option line = "@" , option ;

modifier line = "@" , modifier , block body , "@END" ;

block body = { N } , { line , { N } } ;

line = modifier line | option line | robot line ;

modifier = allowed modifiers , [arg] ;
option = allowed options , [arg] ;

arg = ? regex: /[ \t]+[^\n]*/ ? ;

N = "\n" ;

allowed options = "NO_KETOP" ;
allowed modifiers = "VERSION" | "OS" | "PREAMBLE" | "BRACKETSTART" | "BRACKETEND" ;
```

It is important to note that *inline whitespace*, i.e. sequences made up of only spaces or tabs, are ignored here. This is useful, as it significantly reduces the size and complexity of the EBNF, since there is no need for our syntax to take whitespace into account, with the exception of *robot lines* and *args*, and the library used for parsing an input defined in a grammer that resembles EBNF, which is specified in detail in the next chapter, also allowing for the option to ignore inline whitespace.

Additionally, it is necessary to define the specific *allowed options* and *allowed modifiers* in the EBNF, as otherwise there would not be a easy and general way to differentiate between options and modifiers.

9.1. Its Components

`start` refers to the entire script. This name was chosen because it matches the name that our library has reserved for this purpose. It is made up of any amount of `line`s and empty lines.

A `line` may either be a `modifier line`, `option line`, or `robot line`

A `robot line` is defined as being any line of text that is not empty, does not start with an `@` character, and that ends with a line break character. Here, a regex is used for this matching.

Both `option` and `modifier` refer to their respective allowed keywords, followed by an optional `arg` (*argument*), which is defined as being a string which begins with an inline whitespace character and ends with the end of the line.

An `option line` is a line starting with `@` and a valid `option`.

A `modifier line` starts with an `@` character, is followed by a modifier, then by a `block body`, then by the sequence `@END`, which, as explained in the chapter above, is to be provided by the preprocessor. A `block body` refers to any amount of `line`s, including or not including empty lines.

`N` is simply used for convenience and is defined to be a single line break character.

Part IV.

Implementation [F, A, X]

10. Introduction [F]

In this part, we explain our own implementation for the Script Assembler in detail. Aside from *Syntax Highlighting*, everything was written in the *Python* programming language.

The following chapters belong to this part:

- Chapter 11: *Preprocessor*. This chapter goes over the *Preprocessor*, which converts significant indentation of a *modifier*'s *block body* into a reserved keyword that is understood by the *Parser*. In Chapter 8, the need for this preprocessor is explained in detail.
- Chapter 12: *Parser*. This chapter goes over the *Parser*, which uses the *Lark* library in combination with a modified form of our EBNF from Chapter 15 to recursively parse an input script to an output script and brackets.
- Chapter 13: *Option/Modifier Format*. Here, our implementation for handling the respective keywords within the parser is explained.
- Chapter 15: *TRP-Integration*. Since our project is used within the TRP project, we explain the changes we made to the existing project to integrate our Script Assembler.
- Chapter 16: *Syntax Highlighting*. Since the existing TRP project's *Angular* frontend uses a code editor known as *Monaco Editor*, we implemented *Syntax Highlighting*, in other words text colorization based on keyword syntax, which we go over in this chapter.
- Chapter 17: *Extensibility*. In this chapter, we explain how the Script Assembler may be extended with additional keywords, give examples to keywords that may be useful in the future, and discuss the limits of our parser.

- Chapter 18: *Database*. Here, we explain the *Mockup Database*, which we built to allow the project to run independently to the proprietary *TestRail* system and to aid with testing.

In the TRP project, the following files and directories make up nearly the entirety of the Script Assembler and its components, where the `.` directory refers to the project's root:

- `./TRP/broker/managers/case/script_assembler`: The core logic of the Script Assembler's preprocessor and parser is defined in this directory. It includes the following files:
  - `preprocessor.py`: Here, the `Preprocessor` class is found alongside its logic. This file is shown and explained in Chapter 11.
  - `parser.py`: This file defines the `parse_script` function, which serves as an entry point to the preprocessor and parser. This is explained in Chapter 12.
  - `formatter.py`: This file includes the `Formatter` class, which includes the core logic of the recursive parsing mechanism of the given script in the form of a *Lark* `Tree` object. We explain this mechanism in detail in Chapter 12.
  - `datatypes.py`: Data classes (i.e. classes with the `@dataclass` annotation) that are used by the parser are found in this file and also explained in Chapter 12.
  - `option_modifier_format.py`: This file is used by the `Formatter` and specifies how each respective keyword should be handled. In Chapter 13, we explain the inner workings of this, and Chapter 17 explains how one might use this file to define keywords that may be needed in the future.
  - `parser_exception.py`: Here, two custom exceptions related to the parser can be found: `ParserSyntaxError` and `ParserInternalError`. They are explained in Chapter 14.
- `./TRP/common/dbfunc.py`: This file includes the `DBSession` class, which implements the same functions as `TRSession` of `testrailfunc.py` (which is found in the same directory) in order to connect to and interact with our *PostgreSQL* mockup database. Chapter 18 explains this in detail.
- `./Angular/src/app/app.module.ts`: This represents the primary configuration file for the *Angular* project, which has been provided by our project partner,

Martin Wieser from KEBA Group AG. It includes the configuration for *Monaco Editor*. Chapter 16 explains how we expanded it to allow for *Syntax Highlighting* for our keywords.

- `./TRP/broker/managers/case/casemanager.py`: This file has been provided by our project partner, Martin Wieser from KEBA Group AG. For our project, the `assemble_robot_script` method is important, as it was modified by us to integrate our Script Assembler into the existing TRP project, which is explained in Chapter 15.

11. Preprocessor [A]

11.1. Introduction

As already established earlier on, the Script Assembler introduces a custom syntax added on top of the Robot Framework. It uses annotations such as `@VERSION` to control the test suit setup. To process this grammar the implementation uses the Lark parsing library [**?**]. However while the chosen indentation-based syntax is very intuitive for the developers, it introduces technical difficulties for the parser.

Indentation-based scoping is context-sensitive. To find out if a line belongs to a block, the parser needs to know the indentation level of the previous annotation. That requires it that the current indentation depth is saved. While the Lark library provides its own indenter utility for parsing whitespace-scoped languages, it processes indentation globally [**?**]. Because the Script Assembler has to differentate between its own language and the contained Robot Framework code, Lark's indenter would generate tokens for Robot Framework code indents which might lead to blocks that close too early.

That is why an additional preprocessing step is required before the Lark parser can evaluate the script. This step is handled by a preprocessor, whose goal is to isolate the meta-language indentation from the content-language indentation before parsing the script.

11.2. Design and Transformation Rules

11.2.1. Content vs. Meta-Language Conflict

The fundamental problem is that both languages use spaces and indentation to define logic and structure. In the Robot Framework, spaces and tabs separate keywords

from arguments, and indentation marks the contents of blocks like `FOR` loops and `IF` statements, as shown with green highlights in Figure 4.

At the same time, the Script Assembler uses indentation to determine which lines belong to a specific modifier, as shown with orange highlights in Figure 4. This leads to the problem of having to somehow seperate those two kind of indents and thus the need for our own Preprocessor instead of the Lark Indenter.

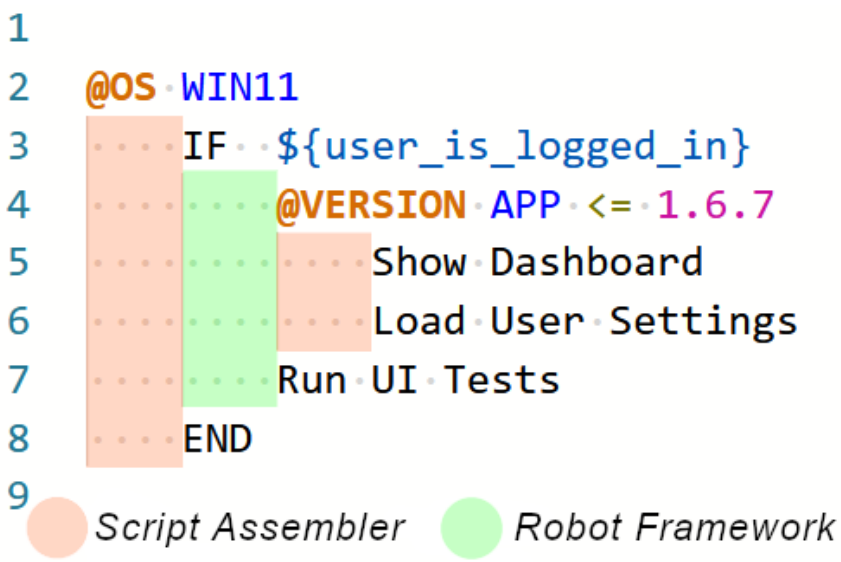


Figure 4.: Difference between meta-language and content-language indents.

11.2.2. Explicit Block Scoping

To make the syntax context-free, the implicit visual scoping provided by indentation must be changed to explicit block scoping. Clear start and end markers must be defined. This is comparable to the use of curly braces `{}` in C# or Java, or to opening and closing tags like `<div>` and `</div>` in HTML.

For each modifier found by the Preprocessor, the start of the block is simply the modifier itself. The end of the block, shown by a decrease in indentation in the raw script. The Preprocessor marks this by inserting the keyword `@END`.

With these clear boundaries, the EBNF grammar in Lark is simplified. It only needs to search for the `@END` token to recognize the end of a block, allowing it to completely ignore the surrounding whitespace.

11.2.3. Preserving Indents

The following example uses several annotations nested within standard Robot Framework control structures. Listing 16 shows a typical raw Preprocessor input script written by a test developer.

Listing 16: Raw input script with nested modifiers

```
1 Setup Test Environment
2     @VERSION BIOS == 1.6.1
3         Log    Checking BIOS Version
4         @OS WIN11
5             Install Driver    audio_win11.sys
6         Restart Node
```

In Listing 16, a Robot Framework keyword (`Setup Test Environment`) is the root scope. Within these block, the outer Script Assembler layer performs a BIOS version check. This block contains a normal Robot command (`Log`) followed by an inner modifier that checks the operating system. This inner modifier contains a command for a driver installation. Finally, a `Restart Node` command is executed within the BIOS block, but outside the inner operating system check.

The Preprocessor should calculate the active layers of modifiers and remove only the leading indents belonging to these specific layers. Listing 17 shows the expected output.

Listing 17: Expected Preprocesssor output

```
1
2 Setup Test Environment
3 @VERSION BIOS == 1.6.1
4     Log    Checking BIOS Version
5 @OS WIN11
6     Install Driver    audio_win11.sys
7 @END
8     Restart Node
9 @END
```

Comparing the two listings shows perfectly how the Preprocessor is supposed to work. The modifier lines themselves (lines 2 and 4 in Listing 17) are completely stripped of their indentation. This is needed since the EBNF grammar only sees modifiers that begin with an `@` symbol, so if it is padded with tabs it is not recognized.

Additionally and most importantly the indentations of the contained language are preserved while the modifier indents are not. For example in the raw text the line `Install Driver audio_win11.sys` was indented three times. Once for the BIOS version check modifier, once for the OS check modifier and once for the actual Robot

Framework block. In the result the two modifier indents are removed, leaving only the Robot Framework one. `Restart Node`

Finally, the decrease of indentation before the `Restart Node` command triggers the Preprocessor to insert the `@END` tag and the open end adds another one lines 6 and 8 in Listing 17).

11.3. Evolution of the Algorithm

Developing an appropriate algorithm for this was a technical challenge. The algorithm has to correctly identify mixed indentation levels and clearly distinguish between a line that defines a new lexical scope and a line that just belongs to an existing scope. Parsing context-sensitive, whitespace-dependent meta-languages is a highly complex process, as demonstrated by the evolution of the Preprocessor logic.

11.3.1. Linear Depth Counting Approach

The first version of the Preprocessor used a simple linear mechanism for counting layers. The basic idea was that the depth of a modifier block directly correlated to a simple integer counter.

In this naive approach, the algorithm maintained a single integer variable called `current_layer`, starting at zero. The Preprocessor iterated through the script line-by-line, searching for the `@` symbol. When it found a modifier, it incremented the `current_layer` counter by one. For all following lines, the algorithm removed a number of leading indents exactly equal to the value of `current_layer`.

To find the end of a block, the algorithm compared the total number of indents on the current line against the `current_layer` variable. If a line had fewer indentations than the active layer count the algorithm assumed that the block had ended. It then inserted an `@END` tag and decremented the counter. This logic worked in simple situations. However, it contained a fundamental flaw regarding relative and absolute indentation depths.

11.3.2. Collision with the Structure of the Content

The linear depth counting approach unfortunately failed when Script Assembler annotations were mixed with Robot Framework commands that use indents inside a modifier. At that point the absolute count of tabs of a line is no longer just determined by the Script Assembler modifier depth.

Listing 18 demonstrates the specific edge case that exposed the flaw in the naive algorithm.

Listing 18: Problematic edge case

```
1 @VERSION BIOS == 1.2.3
2     FOR    ${item}    IN    @{ITEMS}
3         @VERSION KBDRIVER != 0.2.5
4             Add Item To Cart    ${item}
5         Sleep    0.5s
6     END
```

When verifying the algorithm with Listing 18, the error occurs in line 5.

1. **First Modifier:** On line 1, the first `@VERSION` modifier sets the `current_layer` to 1.
2. **Inner Modifier:** On line 3, the inner `@VERSION` modifier increments the `current_layer` to 2.
3. **Conflicting Line:** On line 5, the `Sleep` command has exactly 2 leading tabs (one from the outer modifier, and one from the `FOR` loop).

The algorithm compares the `Sleep` line's tab count (2) to the `current_layer` (2). Since the numbers are equal, the algorithm assumes that the `Sleep` command is still in the inner `@VERSION` block. But this is wrong, since the inner `@VERSION` block started with two tabs, thus requiring any code inside of it to have 3 tabs. This means that the `Sleep` command actually exits the inner block.

That is why the algorithm unfortunately fails, as seen in Listing 19. It is counting tabs in `current_layer` integer for the indentation and completely ignores how many of these tabs were actually introduced by the contained language.

Listing 19: Wrong output of the problematic script with the first approach

```
1  @VERSION BIOS == 1.2.3
2  FOR    ${item}    IN    @{ITEMS}
3  @VERSION KBDRIVER != 0.2.5
4      Add Item To Cart    ${item}
5  Sleep    0.5s
6  @END
7  END
8  @END
```

11.3.3. Stack-Based Approach

To fix this collision, the algorithm required a different approach. Instead of tracking a simple depth integer, it had to explicitly store the absolute indentation level at the exact moment each modifier scope was opened.

The revised Preprocessor uses a stack to keep track of the indents. Whenever the Preprocessor finds a new modifier, it calculates the exact number of leading tabs on that line and pushes this absolute tab count onto the top of the stack.

This approach maps the nested structure of the text to a linear state history. The value at the top of the stack is always the baseline tab count of the currently active innermost modifier.

Re-evaluating Listing 18 using this stack-based approach yields the correct behavior:

- **Line 1:** Outer modifier has 0 tabs. Stack becomes: `[0]`.
- **Line 3:** Inner modifier has 2 tabs. Stack becomes: `[0, 2]`.
- **Line 4:** Content has 3 tabs. The value 3 is greater than 2 (top of stack). The content belongs to the inner block.
- **Line 5:** The `Sleep` command has 2 tabs. The value 2 is less than or equal to 2 (top of stack).

Listing 20: Correct output of the problematic script with the final approach

```
1  @VERSION BIOS == 1.2.3
2  FOR    ${item}    IN    @{ITEMS}
3  @VERSION KBDRIVER != 0.2.5
4      Add Item To Cart    ${item}
5  @END
6      Sleep    0.5s
7  END
8  @END
```

When the algorithm processes line 5, it sees that the current tab count (2) is less than or equal to the top of the stack (2). This triggers the scope resolution logic. The

Preprocessor removes the top value from the stack and correctly inserts the `@END` tag to terminate the inner `@VERSION` block. Finally, it reaches the end of the file, and since it still has one modifier to close, it ends the output with a final `@END`.

By using a stack, the Preprocessor can perfectly keep track of at which level a modifier starts and also where it should end. This makes it entirely unaffected of whatever indentation structures the content uses.

11.4. Implementation Details

The theoretical concepts and the stack-based state machine were implemented in Python. The resulting `Preprocessor` class encapsulates the state variables, the regular expressions for token identification, and the transpilation loop that processes the script line-by-line.

11.4.1. Initialization and Utilities

The Preprocessor is initialized with specific regular expressions. These expressions identify valid modifiers and are passed to the Preprocessor by the Parser, removing the need to maintain them in two places. Furthermore, the class defines utility functions to manage the indentations. Listing 21 shows this class initialization alongside these core utility functions.

An important detail in Listing 21 is the definition of `indent_regex`. Different code editors and Integrated Development Environments (IDEs) format files differently. The regular expression `r"^(?:\t| {4})+"` handles this reliably. It matches any continuous sequence of leading whitespace. It strictly treats either a single tab character or exactly four spaces as one logical unit of indentation.

This regular expression is then used by two utility functions. First, the `count_indents` method calculates the indentation depth of a line. It sums the `\t` characters and blocks of four spaces. This process reduces the visual indentation into a single integer.

Second, the `remove_indents` method removes the tabs of the meta-language while preserving the tabs of the content. It stores all indentations into an array. After that, it removes the first elements based on the `amount` parameter. This parameter corresponds

Listing 21: Class initialization and indentation utility functions

```python
import re
from broker.managers.case.script_assembler.parser_exception import
    ParserSyntaxError
from logging import Logger

log: Logger = Logger("SCRIPT ASSEMBLER")

class Preprocessor:
    modifier_regex: str
    option_regex: str
    indent_regex: str = r"^(?:\t| {4})+"

    def __init__(self, modifier_regex: str, option_regex: str) -> None:
        self.modifier_regex = modifier_regex
        self.option_regex = option_regex

    def count_indents(self, line: str) -> int:
        """Counts the number of indents at the start of a line."""

        match = re.match(self.indent_regex, line)
        if not match:
            return 0

        indent_str = match.group()

        # Count number of indents
        tabs: int = indent_str.count('\t')
        spaces: int = indent_str.count('    ')

        return tabs + spaces

    def remove_indents(self, line: str, amount: int) -> str:
        """Removes 'amount' indents from the beginning of a line."""

        match = re.match(self.indent_regex, line)
        if not match:
            return line  # No indents to remove

        indent_str = match.group()
        indents = re.findall(r"\t| {4}", indent_str)

        # Remove x indents
        remaining_indents = ''.join(indents[amount:])
        rest_of_line = line[len(indent_str):]

        return remaining_indents + rest_of_line
```

to the current depth of the modifier stack. Finally, it restores the line by puts back together the remaining indents with the actual code.

11.4.2. Main Processing Loop

The `preprocess` method handles the line-by-line transformation. Listing 22 shows the initialization of the state tracking variables: the `result` string buffer, the `layer_tabs` stack, and the `is_modifier_empty` flag.

Listing 22: Initialization of the main preprocessing loop

```
46
47      def preprocess(self, script: str) -> str:
48
49          log.info("Starting preprocessing of robot script...")
50          result: str = ""
51          layer_tabs: list[int] = []
52          is_modifier_empty: bool = False
53
54          for line in script.splitlines():
55              stripped: str = line.strip()
56              if stripped == "":
57                  continue
58
59              log.debug(f"Processing robot script line: \"{stripped}\"")
60
61              mod: bool = False
62
63              if stripped.startswith("@"):
64                  if stripped == "@END":
65                      raise ParserSyntaxError("Cannot have @END in script since it
                            is an internal keyword")
66                  mod = re.fullmatch(self.modifier_regex, stripped.split('
                        ')[0][1:]) is not None
67
68              # [...]
```

First, the loop skips any empty lines. Before using the slower `re.fullmatch` operation, the algorithm checks is the line starts with an `@` symbol and thus is a modifier. This filters out the majority of lines.

If the line begins with that symbol, we check if the modifier is `@END`. Since this is an internal token only used for terminating the blocks, users are not allowed to use it in raw scripts. That means if it exists in the code, we raise a `ParserSyntaxError`. This should basically never happen, since the intermediate code is never exposed to the user and `@END` tags are not highlighted by the later described syntax highlighting (see Chapter 16 or Figure 10).

Finally, the algorithm checks if the modifier is valid by matching it against the `modifier_regex` passed by the Parser. If it is valid, the `mod` flag is set to `True` for later use.

11.4.3. Scope Handling

After reading a line, the Preprocessor decides whether the line belongs to the current scope, opens a new scope, or closes existing scopes. Listing 23 illustrates this stack management logic.

Listing 23: The stack implementation for resolving indentation scopes

```
69
70              # [...]
71              if len(layer_tabs) > 0:
72                  tabs: int = layer_tabs[-1]
73                  cnt: int = self.count_indents(line)
74
75                  if mod:
76                      line = stripped # don't want any indents on modifier lines
77
78                  if cnt <= tabs:
79                      while len(layer_tabs) > 0 and cnt <= layer_tabs[-1]:
80                          layer_tabs.pop()
81                          result += "@END\n"
82
83                          if is_modifier_empty:
84                              raise ParserSyntaxError("Cannot have empty modifiers")
85
86                  result += self.remove_indents(line, len(layer_tabs)) + "\n"
87
88                  if mod:
89                      layer_tabs.append(cnt) # adds count of tabs to the stack
90                      log.debug(f"Processing modifier; Increasing layer by one")
91                  is_modifier_empty = False
92
93              else:
94                  result += f"{line}\n"
95
96              # [...]
```

If the stack is not empty, the algorithm compares the absolute indentation count of the current line (`cnt`) with the top value of the stack (`tabs`). If the current count is less than or equal to the top value, a scope ends.

This resolution is implemented using a `while` loop rather than a simple `if` statement. This ensures proper handling when a single line with fewer indents closes several nested modifier blocks simultaneuosly. For each level removed, the string `@END\n` is appended to the result.

The empty modifier validation occurs inside this loop. If a block is closed but the `is_modifier_empty` flag is still set to `True` because no inner content was processed, transpilation aborts. This is needed since a modifier without code to modify is pointless.

That is why the Preprocessor assumes that it is most likely either not correctly indented or a typo by the test developer and needs to raise an `ParserSyntaxError` to communicate that. Finally, the `remove_indents` utility is called to strip the indents of the meta-language before the line is appended to the final output stream.

11.4.4. Finalization

At the end of the loop, new scopes are initialized, and the output is verified before being passed to the Parser, as seen in Listing 24.

Listing 24: Handling the root scope and finalizing the output stream at the end of the file.

```
97
98              # [...]
99
100             if mod:
101                 is_modifier_empty = True
102
103                 if len(layer_tabs) == 0:
104
105                     # If there aren't any previous modifiers, add initial root
                            layer
106                     layer_tabs.append(0)
107
108
109         # Close any remaining open scopes
110         if len(layer_tabs) != 0:
111
112             for _ in range(0, len(layer_tabs)):
113                 result += "@END\n"
114
115         log.info("Finished preprocessing the robot script!")
116
117         return result.replace('\r', '')
```

If the current line is a modifier and the stack is empty, a root level of `0` is pushed to the stack to serve as the absolute baseline.

After iterating through the entire file, the algorithm does a final cleanup. If the file ends without returning to zero indentations, the stack still contains unclosed modifier blocks. The logic iterates through all remaining items in the `layer_tabs` stack, adding an `@END` tag for each. Finally, carriage returns are normalized to avoid cross-platform compatibility issues.

11.5. Summary

The Preprocessor acts as a hidden translation layer between the human-readable script and the more machine-centric requirements of the Parser. By using a stack the algorithm properly stores and processes the indentations of the meta-language. It removes the assembler's scoping indicators while retaining the precise indentation required for the Robot Framework runtime environment. Furthermore, the Preprocessor also handles the first validation by rejecting manually added `@END` keywords and empty modifiers.

12. Parser [F]

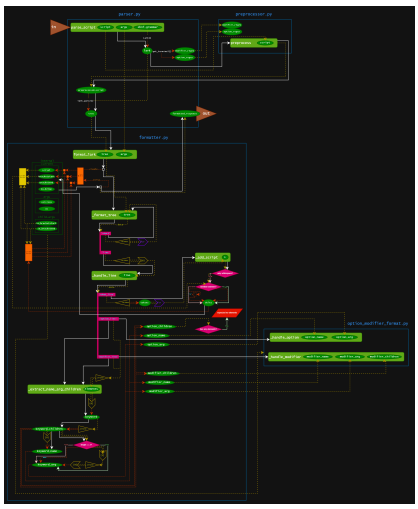
12.1. Introduction

In this chapter, we will focus on the *parser*, which is the part of our project that reads the input sequence given by the preprocessor alongside context information and outputs a so-called `FormattedResponse`.

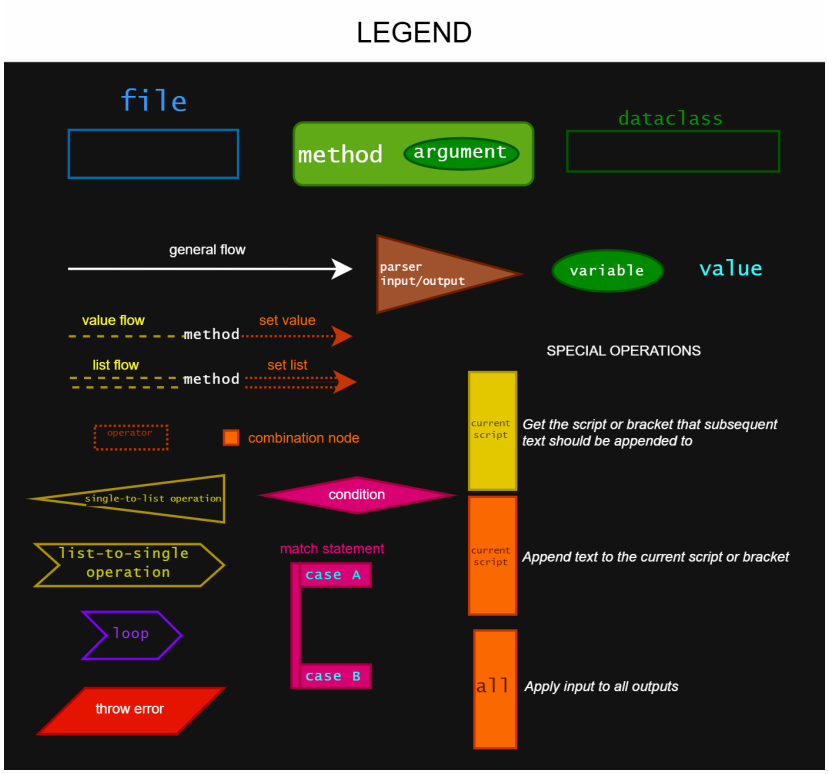
It is written in Python and relies heavily on the Lark library.

The parser recursively works its way through the `Tree` object given by Lark, and iteratively builds up the script, brackets, and options for its output.

Figure 5 shows a flowchart diagram of the entire parser, where the parts that are discussed in this section are depicted in detail.



(a) Downsized flowchart diagram of the parser.



(b) Legend of the parser flowchart diagram.

Figure 5.: Full flowchart diagram of the parser with its legend.

It is important to note that while Figure 5a is at a size where it may not be possible to make out details, subsequent sections of this chapter include enlarged variants, as seen in Figures 6, 7, 8, and 9.

12.2. High-Level Overview

In a nutshell, the parser takes an input in the form of a script as a string, a few contextual arguments as `FormatterArgs`, and a Lark grammar, which resembles an EBNF grammar, as a string, and outputs a `FormattedResponse`, as is evident by its method header:

Listing 25: Method header of `parse_script`.

```
def parse_script(script:str, args:FormatterArgs,
    ebnf_grammar:str)->FormattedResponse:
```

12.2.1. Input: Script

The script is the the raw Robot script that is to be parsed. Importantly, this input should not already have gone through the preprocessor step mentioned in Chapter 11, as it is done in the function.

12.2.2. Input: Arguments

In the method header, `formatter_args` refers to an instance of the `FormatterArgs` class, which is defined to be the following:

Listing 26: Class definition of `FormatterArgs`.

```
@dataclass
class FormatterArgs:
    versions:dict[str, str] # label -> version
    os:str|None
```

This is used for providing the contextual input, which, in our case, is needed to evaluate the conditions specified in the `@VERSION` and `@OS` modifiers.

Here, `versions` should be a dictionary object that maps the label of a module to its actual version on the KeTop. In the actual implementation of `@VERSION`, the operation specified in the argument will be checked against the version provided here for the respective module.

`os` refers to the operating system of the KeTop as a string, or `None` if one is not provided or present (as is generally the case when, for instance, the `no_ketop` flag is set). In the actual implementation of `@OS`, the block body will only be included if the operating

system specified in the argument equals the operating system provided here. If no operating system is provided, `@OS` will always discard its block body.

12.2.3. Input: Lark grammar

The Lark grammar, in the method header labeled as `ebnf_grammar`, represents "a list of rules and terminals that together define a language"[?] as a string. While the Lark library uses its own syntax, it is based on EBNF [?].

In our project, the following string, which is a modified form of the our EBNF to abide by Lark's expected syntax, gets used:

Listing 27: Our EBNF defined using Lark's syntax.

```
start: N* (line N*)*

robot_line: /[^(@|\n)].+?(?=\n)/

option_line: "@" option

modifier_line: "@" modifier ( N )* ( line N* )* "@" "END"

line: modifier_line | option_line | robot_line

modifier: ALLOWED_MODIFIERS arg?
option: ALLOWED_OPTIONS arg?

arg: /[ \t]+[^\n]*/

N: "\n"

%import common.WS_INLINE
%ignore WS_INLINE

ALLOWED_OPTIONS: "NO_KETOP"
ALLOWED_MODIFIERS: "VERSION" | "BRACKETSTART" | "BRACKETEND" | "OS" | "PREAMBLE"
```

The following is changed in listing 27 compared to listing 15:

- Grammar definition syntax is changed from `=` to `:` and the seperator character between terminals (`,`) and the end-of-rule character (`;`) are omitted.
- Repetition is changed from using curly braces (`{...}`) to an asterisk (`...*`). It is important to note that, while it is not needed for our EBNF, Lark also allows for the definition of *one or more* using a plus sign (`...+`) instead.
- Optionals are changed from using square brackets (`[...]`) to `...?`.
- regex tokens are written using slashes (`/.../`).
- Terminals (here `N`, `ALLOWED_OPTIONS`, and `ALLOWED_MODIFIERS`) are defined using uppercase.

- The verbose definition of the `block body` rule is omitted, and it is instead defined directly within a `modifier_line`.
- Inline whitespace is being explicitely ignored using `%import common.WS_INLINE` and `%ignore common.WS_INLINE`.

It is important to mention that, while it might be possible to provide a different grammar, it is very likely to cause issues given the strict adherence of our parser to the above grammar.

12.2.4. Output: FormattedResponse

As long as no exceptions occur, the output of the `parse_script` function will be an instance of the `FormattedResponse` class, which is defined to be the following:

Listing 28: Class definition of `FormattedResponse`.

```
@dataclass
class FormattedResponse:
    script:str
    preamble_dict:dict[int, str]
    bracketstart:str
    bracketend:str
    no_ketop:bool
```

Here, `script` refers to the general part of the script that is not part of a *bracket*. The term *bracket* refers to a block body of either `@PREAMBLE`, `@BRACKETSTART`, or `@BRACKETEND`. If a Robot script with no Script Assembler keywords is provided as input, aside from possible minor whitespace changes, it will be returned without edit in `script`.

`preamble_dict` is a dictionary that maps the block bodies of `@PREAMBLE` modifiers to the number given by their respective arguments.

`bracketstart` and `bracketend` contain the block bodies of `@BRACKETSTART` and `@BRACKETEND` respectively.

`no_ketop` represents whether or not the `no_ketop` flag is enabled; i.e. whether or not the input script contains the `@NO_KETOP` modifier.

12.3. Lark

Lark is a parsing library for Python which can parse any context-free grammar to an object known as a *tree* [?].

After preprocessing, the Script Assembler uses the input script on Lark's `parse` method to get a `Tree` object which, amongst others, includes two important properties: `data` and `children`.

A `Tree` object represents a rule with a name, for example, the outermost rule of a parsed script being `"start"`, alongside, if present, its children, which, in the example of `"start"` are `"line"`s. A *child* is commonly a `Tree` as well, however it may also be a `Token` [?].

According to Lark's official API reference, a `Token` is described as the following:

> "A string with meta-information, that is produced by the lexer. When parsing text, the resulting chunks of the input that haven't been discarded, will end up in the tree as Token instances. The Token class inherits from Python's `str`, so normal string comparisons and operations will work as expected."[?]

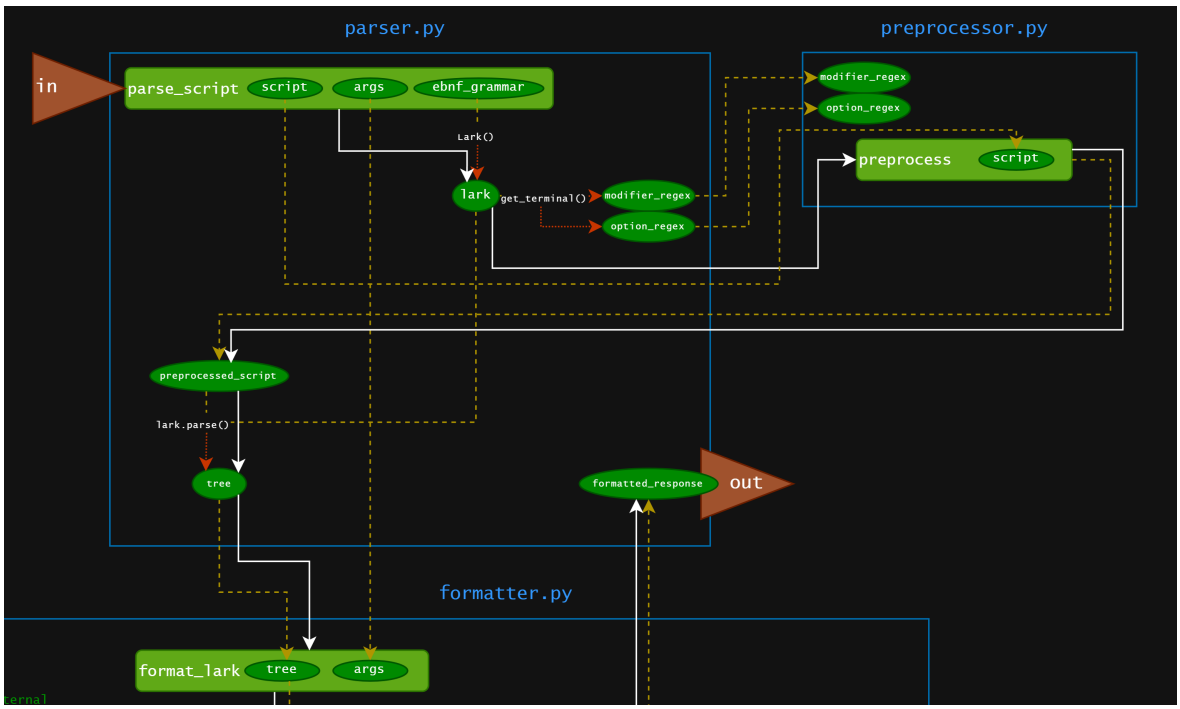
In our parser, we mainly work with `Tree`s to differentiate between lines and keywords, and `Token`s for the contents of robot lines and keyword arguments.

12.4. General Procedure

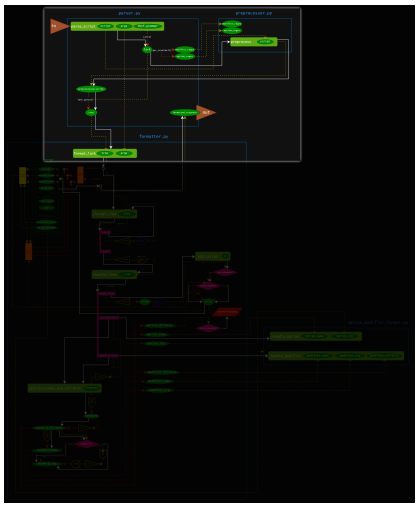
The `parse_script` starts by creating an instance of the `Lark` class using the provided grammar. Using Lark's `get_terminal` function, the allowed modifiers and options as specified in the grammar are fetched and, together with the input script, passed to the preprocessor.

Afterwards, using Lark's `parse` method, the preprocessed script is then converted to a `Tree` object. This is passed to the `format_lark` function, which utilizes the provided `FormatterArgs` to turn it into a `FormattedResponse`, which is then returned.

Figure 6 highlights this procedure.



(a) The detailed part of the parser's flowchart diagram which illustrates the general procedure.



(b) The location of the detailed inset within the full flowchart as seen in Figure 5a.

Figure 6.: The part of the flowchart diagram from Figure 5 that details the general procedure of the parser.

12.5. Formatter

After a script has been preprocessed and parsed by Lark, the `Tree` object is given to the method from the `Formatter` class with the following header:

Listing 29: Method header of `format_lark`.

```
1 def format_lark(self, tree:Tree, args:FormatterArgs)->FormattedResponse:
```

The `Formatter` class recursively parses the `tree`'s children to eventually return a `FormattedResponse`.

12.5.1. FormatterInternal

The formatter keeps track of the current state of the parsing using an internal property named `current` that is of type `FormatterInternal` which is defined as the following:

Listing 30: Class definition of `FormatterInternal`.

```
@dataclass
class FormatterInternal:
    current:FormattedResponse
    args:FormatterArgs
    child_args:ChildArgs
```

This `FormatterInternal` instance is unique per formatting operation and it is passed to any functions that need the context of the formatter. `args` represents the instance of `FormatterArgs` that was provided to the parser that includes information on versions and operating system.

When the `format_lark` function finishes without error, it returns the `current` property of the `FormatterInternal` instance it uses to keep track.

12.5.2. ChildArgs

`child_args` is used for modifiers and their recursive nature. When a line belonging to the block body of a modifier is handled, it may need to be handled differently depending on factors such as whether or not that line should be part of a bracket.

When a `modifier_line` is hit, the function handling that modifier may return a new instance of `ChildArgs`. Then, this instance will be used for all lines that belong to the block body of that modifier.

The `ChildArgs` class is defined to be the following:

Listing 31: Class definition of `ChildArgs`.

```
@dataclass
class ChildArgs:
    preamble_index:int = 0
    in_bracketstart:bool = False
    in_bracketend:bool = False
```

Here, `preamble_index` represents the index of the preamble bracket that new lines should be appended to. If this variable is set to `0`, it means that the formatter is currently not inside of an `@PREAMBLE` modifier.

`in_bracketstart` and `in_bracketend` represent whether or not the formatter is currently within a respective `@BRACKETSTART` or `@BRACKETEND` modifier. If either of this is the case, new lines will be appended to the respective `bracketstart` or `bracketend` brackets, respectively.

Here, `preamble_index`, `in_bracketstart`, and `in_bracketend` are mutually exclusive to one another, i.e. at most only one of them can be *active* at a time.

12.5.3. Line handling

After the formatter is past the `"start"` entry point of the initial `Tree`, it will run `dle_line` on each line element, which is a function with the following header:

Listing 32: Method header of `_handle_line`.

```
1  def _handle_line(self, line:Tree)->None:
```

This function differentiates whether the `line` matches `"robot_line"`, `"option_line"`, or `"modifier_line"` in the following way. Note that all functions listed below will be explained shortly.

- For `"robot_line"`: Since Robot lines are made of, for the purposes of the parser, arbitrary text, they can be added to the output as-is using the `_add_script` function.
- For `"option_line"`: The function `OptionModifierFormat.handle_option` is called with the current `FormatterInternal`, the option's name (e.g. `"NO_KETOP"`), and, if present, the argument string.
- For `"modifier_line"`: The function `OptionModifierFormat.handle_modifier` is called with the current `FormatterInternal`, the modifier's name (e.g. `"VERSION"`), if present, the argument string (i.e. `BIOS > 1.0.0`), and its children. The children represent the modifier line's block body. Then, the output of this function is given to the `_format_children` function for further processing.

`_add_script` adds an input string to the output script or respective bracket. This is the function that appends text (Robot lines) to the final `FormattedResponse`. Where

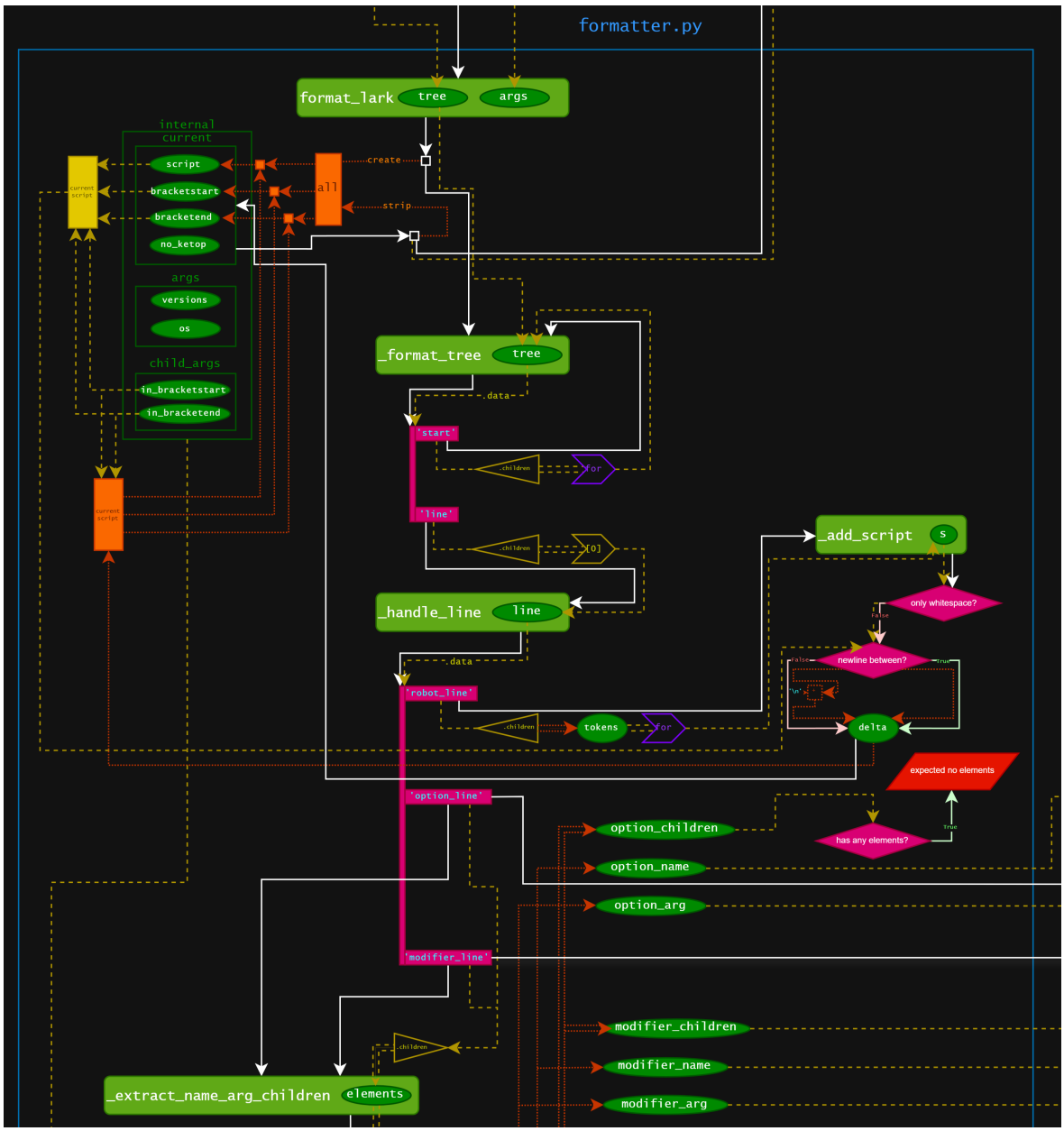
exactly the text gets appended to depends on the state of the current `child_args`. If `child_args` specifies that the formatter is currently within a bracket, it will be appended to `bracketstart`, `bracketend`, or the given index of `preamble_dict`, respectively; otherwise, it will be appended to `script`.

`OptionModifierFormat.handle_option` is part of a seperate class to handle the specific option and, in turn, alter flags of the `current` (`FormattedResponse`) provided in the `FormatterInternal` instance.

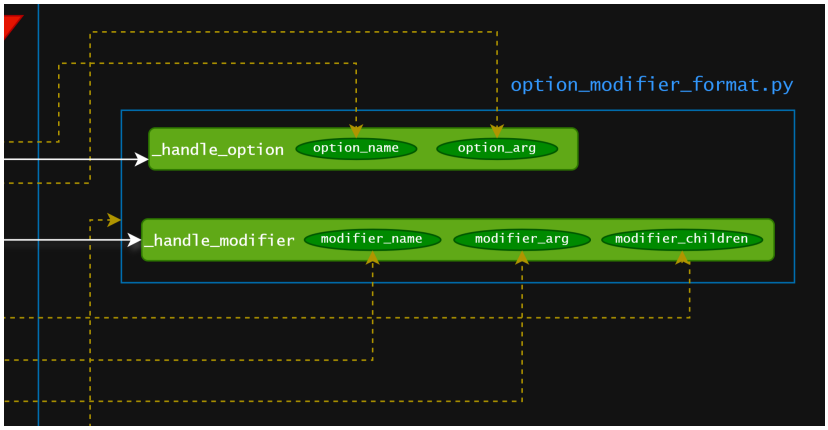
`OptionModifierFormat.handle_modifier` is part of a separate class to handle the specific modifier. Unlike `OptionModifierFormat.handle_option`, this function has return values: firstly, its children (which, as stated above, represent the block body) and the new instance of `ChildArgs` to be used as context information for the lines in its block body.

`_format_children` sets the formatter's current `ChildArgs` to the instance given as parameter, runs `_handle_line` on each element of the provided list of `Tree`s, and then changes the formatter's current `ChildArgs` value back to what it was at the beginning of this function call. This way, a modifier's `ChildArgs` only gets used for its children (block body). If a modifier does not need to pass custom `ChildArgs` to its children, it may return the `ChildArgs` of `current` (`FormatterInternal`), as opposed to a new instance.

Figures 7 and 8 highlight the precise procedure to recursively handle each line.



(a) The detailed part of the parser's flowchart diagram which illustrates how the main part of the formatter recursively handles lines.



(b) The detailed part of the parser's flowchart diagram which shows how arguments are passed to Option/Modifier Format.

Figure 7.: Detailed insets for the parser flowchart shown in Figure 5.

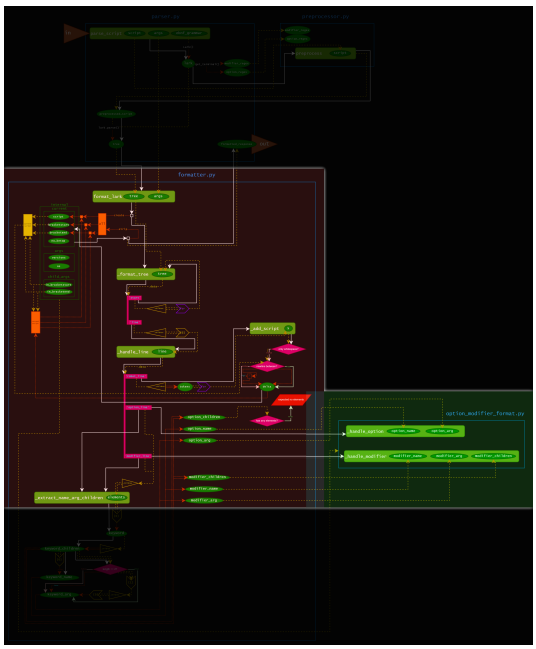


Figure 8.: Location of the detailed insets from Figure 7 within the full flowchart (Figure 5a).

12.6. Extracting name, arg, children

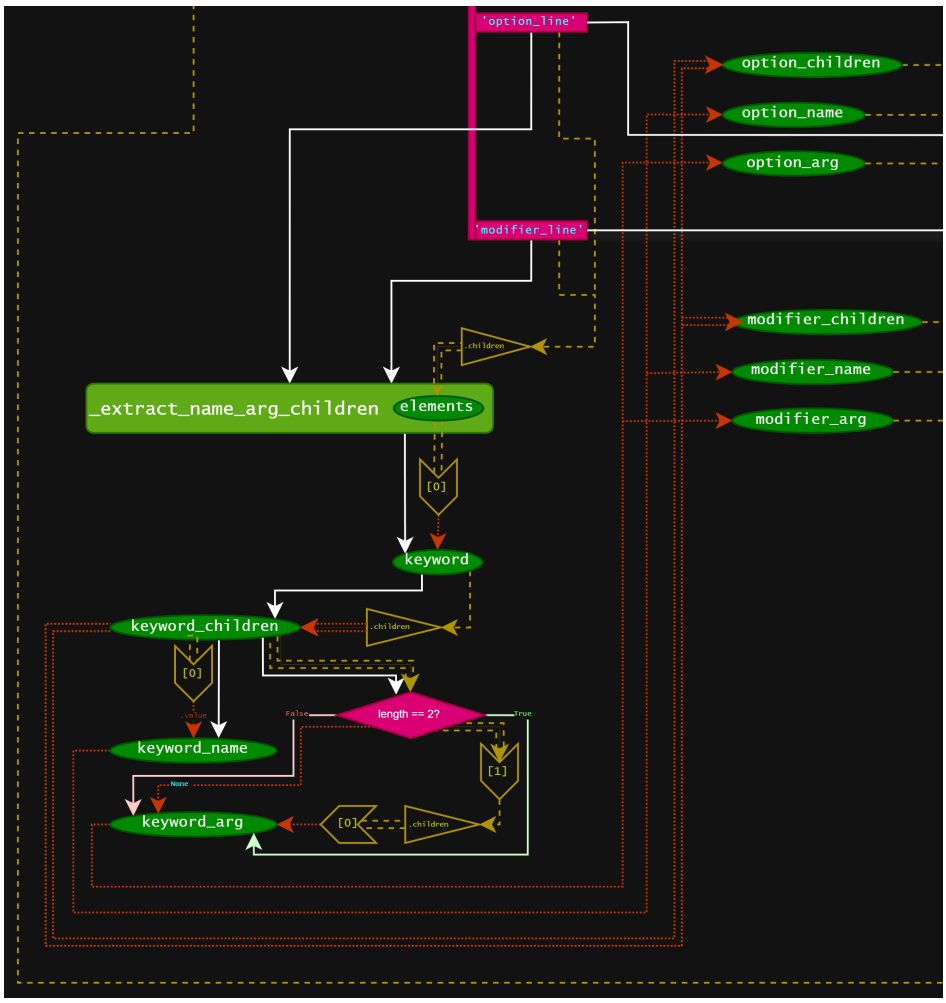
A keyword's name, its argument if present, and its children (block body) get extracted from the `_extract_name_arg_children` function. In the case of an option, the expected amount of children is zero.

In both cases however, the `children` of the line get passed to this function. Assuming the input to be in the correct general format, the following can be said about the values of the elements of this list, which will be referred to as `elements`:

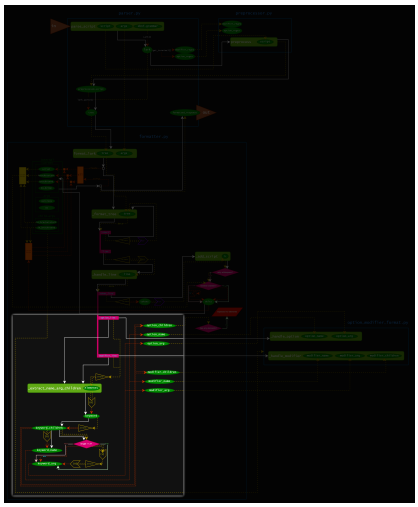
- `elements[0]`: This represents the keyword as option or modifier.
  Its `$.data` is either `"option"` or `"modifier"`.
- `elements[0].children`: This list either contains one or two elements.
- `elements[0].children[0]`: This element is a `Token` and its `$.value` represents the name of the keyword (e.g. `"VERSION"`).
- `elements[0].children[1]`: If present (i.e. if `elements[0].children` contains two elements), its `$.children[0].value` represents the argument. Otherwise, there is no argument.
- `elements[1:]`: The rest of the `elements` list, i.e. everything but the first element, is considered the children (block body) of the modifier.

Here, `$` is used as a placeholder for the respective variable from before the colon of each bullet point.

Figure 9 highlights this procedure.



(a) The detailed part of the parser's flowchart diagram which illustrates the extract-name-arg-children procedure.



(b) The location of the detailed inset within the full flowchart as seen in Figure 5a.

Figure 9.: The part of the flowchart diagram from Figure 5 that details the extract-name-arg-children procedure of the parser.

13. Option/Modifier Format [F]

This chapter highlights the implementiation of how options and modifiers are handled by the parser.

The `OptionModifierFormat` class is used to handle specific keywords. It includes two main functions, which are `handle_option` and `handle_modifier`. Each of these functions matches for its respective name.

13.1. Option @NO_KETOP

When encountered, the `no_ketop` variable gets enabled in the `current` (`FormattedResponse`).

If that variable has already been set to true (i.e. if `@NO_KETOP` was used twice in the file) a warning will be logged.

The `@NO_KETOP` option expects no argument.

13.2. Modifier @VERSION

The `@VERSION` modifier represents a conditional statement and requires an argument.

The argument must be in the form `<component> <comparison> <version>`, where `<component>` is the label of the component that is being checked, `<comparison>` is a comparison operator (either `>`, `<`, `>=`, `<=`, `==`, or `!=`), and `version` is the version to compare the provided component version with, in the *Semantic Versioning* scheme.

The component together with the comparison operator and version will be checked against the `versions` variable provided in `FormatterArgs`, which is a dictionary where

the key represents the actual component together with its version on the KeTop. This comparison is case-insensitive and ignores any leading or consecutive whitespace.

If provided component from the argument is not found in the `versions` dictionary variable, a warning will be logged and the body gets discarded.

If the comparison is satisfied, the body will be further recursively parsed; otherwise, the body will be discarded.

The comparison works using the `_compare_versions` function with the following method header:

Listing 33: Method header of `_compare_versions`.

```
1 def _compare_versions(self, actual_version:str, operator:str, compare_version:str)
      -> bool:
```

Here, the `semantic_version` library is used. *Semantic Versioning*, as discussed in Chapter 5, is a standard for assigning and incrementing version numbers. [**?**]

Both the string provided from the `versions` dictionary variable and the one from the argument are attempted to be converted to instances of `semantic_version`'s `Versions` class using its constructor.

This fails when either of the strings is not in a valid *Semantic Versioning* format. In that case, a warning is logged, and `semantic_version`'s `Version.coerce` method is attempted to be used instead to convert it into a `Version` instance.

According to `semantic_version`'s API reference, the rules for `Version.coerce` are as follows:

- "If no minor or patch component, and `partial` is `False`, replace them with zeroes
- Any character outside of `a-zA-Z0-9.+-` is replaced with a `-`
- If more than 3 dot-separated numerical components, everything from the fourth component belongs to the `build` part
- Any extra `+` in the `build` part will be replaced with dots"[**?**]

If this fails as well, a `ParserSyntaxError`, which is explained in Chapter 14, is raised.

As `semantic_version` utilizes operator overloading, both versions can be compared using the operators `>`, `<`, `>=`, `<=`, `==`, or `!=` as if they were numbers, where a newer version is considered to be *greater than* an older version. [**?**].

A `match` statement is used on the operator string from the argument to choose the corresponding comparison between the version given in the `versions` dictionary and the version from the argument. If an unsupported or otherwise invalid operator is given, a `ParserSyntaxError` is raised.

13.3. Modifier @OS

The `@OS` modifier represents a conditional statement and requires an argument.

The argument represents the operating system to check; it is compared against the `os` variable provided in `FormatterArgs`. This comparison is case-insensitive ignores any leading whitespace.

If the provided `os` variable is `None`, a warning will be logged and the body gets discarded.

If the comparison is satisfied, the body will be further recursively parsed; otherwise, the body will be discarded.

13.4. Modifier @PREAMBLE

The `@PREAMBLE` modifier continues to recursively parse the body text, however the parsed text will be appended to the respective index of `preamble_dict`, as opposed to `script`. This index is given by the required argument.

This is done by returning a `ChildArgs` instance with the `preamble_index` variable set to that given by the argument.

A syntax exception will be thrown if it is defined inside `@BRACKETSTART`, `@BRACKETEND` or another instance of `PREAMBLE`.

13.5. Modifiers @BRACKETSTART and @BRACKETEND

The `@BRACKETSTART` or `@BRACKETEND` modifier continue to recursively parse the body text, however the parsed text will be appended to `bracketstart` or `bracketend` respectively, as opposed to `script`.

This is done by returning a `ChildArgs` instance with `in_bracketstart` or `in_bracketend` respectively set to `True`.

A syntax exception will be thrown if it is defined inside `@BRACKETSTART`, `@BRACKETEND` or `@PREAMBLE`.

The `@BRACKETSTART` and `@BRACKETEND` modifiers expect no argument.

14. Exceptions and Logging [F]

This chapter will explain how the preprocessor and parser handle exceptions and logging.

14.1. Logging

In programming, logging is the process of recording information during the execution of a program. It is commonly considered to be an important part of an application, because it allows for issues to be diagnosed and debugged, monitoring performance and application health, analyzing usage patterns and more [**?**].

In Script Assembler, most things are logged using Python's `logging` module, which had already been set up to work in the TRP project before the Script Assembler project was started on.

This module allows for logging on multiple *levels* [**?**]. The levels that we utilized in Script Assembler are the following:

- `debug`: We use this level to log nearly every step that the parser takes. As the name suggests, it is useful for debugging purposes, when it might be crucial to consider the exact steps that were taken.
- `info`: We use this level for anything that might be *useful to know* regarding the overall flow of the program, that is not critical or necessarily important to consider. It is used, for instance, when the body from a conditional modifier, such as `@VERSION`, is discarded.
- `warning`: We use this level to notify the user about situations that, despite not being exceptions, might not be intentional or entirely logical, and could therefore have unforseen consequences. It is used, for instance, when an option that merely

sets a single flag to a value, like `@NO_KETOP`, is used multiple times in the same script.

In this section, the only log level we will be taking an in-depth look at in this thesis is `warning`, as it may provide insight into certain edge cases.

14.1.1. Warnings

The following is a list of all warnings of `OptionModifierFormat` a user may run into when using the parser:

- *NO_KETOP flag has already been set for this file*: This warning is raised when the `@NO_KETOP` option is used and the `no_ketop` variable is already set to true from a previous use of the option in the same file. In this case, the flag remains enabled.
- *Version for {...} is not provided. Discarding block*: This warning is raised when the `@VERSION` modifier is called and the module given in the argument has not been provided in the `versions` dictionary of `FormatterArgs`. That modifier's body gets discarded.
- *Provided OS is None. Discarding block*: This warning is raised when the `@OS` modifier is used and the `os` variable o f `FormatterArgs` is `None`. That modifier's block gets discarded.
- *Provided actual version ({...}) is not in a valid semantic version format (https://semver.org/) trying to force parse...*: This warning is raised when the `@VERSION` modifier is called with an argument with a version that is not a valid semantic version format. When this happens, `semantic_version`'s `Version.coerce` function is used, which attempts to correct the version string. If this fails, a `ParserSyntaxError` is raised.

Many of the warnings above were requirements given by our project partner.

14.2. Exceptions

In programming, when the normal flow of a program is disrupted, an exception often gets thrown. Aside from built-in exceptions like after attempting to divide a number

by zero, Python also allows for defining custom exceptions. This can be useful to, for instance, improve the clarity of error messages, make it easier to diagnose issues, and to avoid code duplication [**?**].

In this chapter, the terms *exception* and *error* are used interchangeably.

The exceptions `ParserSyntaxError` and `ParserInternalError` have been created for use in the parser and the preprocessor, and they are defined as the following:

Listing 34: Class definitions of `ParserSyntaxError` and `ParserInternalError`.

```
1  class ParserSyntaxError(Exception):
2      message:str
3      def __init__(self, message)->None:
4          super().__init__(message)
5          self.message = message
6
7  class ParserInternalError(Exception):
8      message:str
9      def __init__(self, message)->None:
10         super().__init__(message)
11         self.message = message
```

These exceptions are specified to be used for the following purposes:

- When the use of a keyword or its related option/modifier syntax does not match specification, or when there is an attempt to use a keyword that is not recognized by the parser, a `ParserSyntaxError` should be thrown.
- When an internal problem prevents the parser or preprocessor from being able to continue in an intended way, a `ParserInternalError` should be thrown. However, it is important to note that, ideally, the code should instead be written in a way that makes getting such exception impossible, as any unusual part of the input text should be handled by specified `ParserSyntaxError`s.

While neither of these exceptions have been used rigorously, many different scenarios have been considered and tested to prevent falsified or otherwise unhelpful results from the Script Assembler.

For a user of our project, it might be important to be aware of the possible exceptions that are intended for them to run into if something is found to be syntactically incorrect. For this reason, what follows is a list of occasions that would raise a `ParserSyntaxError`:

- Usage of the `@END` keyword which is reserved for internal purposes.
- Defining a modifier with no body.
- Usage of a keyword that the parser does not recognize.

- Omitting the argument for a keyword that requires one. This includes `@VERSION`, `@OS`, and `@PREAMBLE`.
- Including an argument for a keyword that does not allow for one. This includes `@NO_KETOP`, `@BRACKETSTART`, and `@BRACKETEND`.
- Defining a `@PREAMBLE`, `@BRACKETSTART`, or `@BRACKETEND` inside of the body of a `@PREAMBLE`, `@BRACKETSTART`, or `@BRACKETEND`.
- The argument of the `@VERSION` modifier not being made up of three parts each seperated by a space.
- The comparator given in the `@VERSION` modifier is not valid.
- If `semantic_versioning` fails to parse a version number to semantic version format after trying to force parse with `Version.coerce`.
- When Lark runs into *unexpected characters* and is therefore not able to match the remaining characters to a terminal [**?**].
- When Lark runs into an *unexpected end of file* when it would still expect a token [**?**].

15. TRP-Integration [X]

The following section will focus on two aspects, namely the usage of the testing environment and the assembling of the final script, which occurs in the background.

15.1. Using The Testing Environment

The user interface of the testing environment has initially been designed to give the user the ability to easily monitor which test cases are present in the system and to quickly run a custom selection of these test cases.

The test cases are displayed in a makeshift file explorer, which organizes the test cases in a hierarchical structure based on their respective test suites and sections. Checkboxes are provided next to each test case, as well as next to each test suite and section, allowing the user to easily select or deselect entire groups of test cases. Once the user selected a batch of test cases, they can give the test run a name and click to start the test run.

Once the test run is started, the Script Assembler is responsible for which test cases should actually be included in the final script and in which order they should be placed. These decisions are based on the presence of the keywords in the test cases, as well as the order in which they are selected by the user. Once the final script is assembled, it is ready to be executed in the testing environment, and the results can be analyzed by the user upon completion.

15.2. Assembling The Script

The *casemanager.py* module is responsible for the handling and management of test cases within the system. Considering the new features and improvements that have been

implemented, this module has undergone significant changes. The following subsections will provide a detailed overview of the modifications made to the *casemanager.py* module, in form of code snippets and explanations.

The function that is responsible for assembling the final script has undergone the most significant changes, considering that it is not only one of the most important functions in the module, but also because our work targets the functionality of this function directly. This is because the new features include keywords such as PREAMBLE, BRACKETSTART and BRACKETEND, which explicitly alter the way the script is assembled. Furthermore, other functionalities have been modified to fit the new features.

15.2.1. Detecting Keywords

Listing 35: Old: Checking for NO_KETOP keyword in script

```
1       if "@NO_KETOP" in script:
2           log.warning("found NO_KETOP in test case.")
3           script = script.replace("@NO_KETOP", "")
4           variables["NO_KETOP"] = True
```

In listing 35, the old code checks for the presence of the NO_KETOP keyword in the script directly, and if it is foung, it is removed from the script and a corresponding variable is set to True. This approach is straightforward and works well enough for the old functionality, but it does not take into account any new features that may be added in the future. Since the expandability of the system is a key aspect of our work, this approach has been modified.

Listing 36: New: Checking for NO_KETOP keyword in script

```
1       if formatted.no_ketop:
2           if variables['NO_KETOP']:
3               log.warning('NO_KETOP flag has already been set for this test case')
4           variables['NO_KETOP'] = True
```

In listing 36, the new code checks for the presence of the NO_KETOP keyword in a more structured way, by using a formatted object that contains all the relevant information about the given script. This includes the presence of a BRACKETSTART or BRACKETEND keyword. This approach allows the functionality of the test script format to be more extendable, as data pertaining to a new keyword can easily be added to the formatted object without having to modify the core logic of the script assembly process.

Additionally, the new code also includes a check to ensure that the NO_KETOP flag has not already been set for the given test case, since that unintended behavior would indicate a potential issue in the system.

Listing 37: Old: Checking for BRACKETSTART and BRACKETEND keyword in script

```
1       script_lines = script.splitlines()
2           if script and "@BRACKETSTART" in script_lines[0]:
3
4               brackets = script.replace("@BRACKETSTART", "").split("@BRACKETEND")
5
6               start += f"\nT{tc.tr_test_id} - {tc.tr_case.title} [BRACKET START]\n"
7               start += brackets[0]
8
9               end += f"\nT{tc.tr_test_id} - {tc.tr_case.title} [BRACKET END]\n"
10              end += brackets[1]
```

The check for the BRACKETSTART and BRACKETEND keywords worked similarly to the NO_KETOP keyword, as shown in listing 37. The script was split into lines. If the first line contained the BRACKETSTART keyword, the script was split into two parts using the BRACKETEND keyword as a delimiter. The first part was added to the start variable, while the second part was added to the end variable. The identical problems that were present in the NO_KETOP keyword check arise here as well.

Listing 38: New: Checking for BRACKETSTART and BRACKETEND keyword in script

```
1       if formatted.bracketstart != '':
2           start += f"\nT{tc.tr_test_id} - {tc.tr_case.title} [BRACKET START]\n"
3           start += self._add_tab(formatted.bracketstart) + '\n'
4       if formatted.bracketend != '':
5           end += f"\nT{tc.tr_test_id} - {tc.tr_case.title} [BRACKET END]\n"
6           end += self._add_tab(formatted.bracketend) + '\n'
```

As shown in listing 38, the bodies of the BRACKETSTART and BRACKETEND keywords are now stored in the formatted object as separate variables after the script has been parsed. The contents of these variables are then added to the start or end variable respectively, if they are not empty.

15.2.2. Ordering of Scripts

The initial implementation of the script assembly consisted of three variables that were used to store the different parts of the script, namely:

- *start*:
  This variable was used to store the scripts that contained the BRACKETSTART keyword.

- *regular*:

  This variable was used to store the scripts that did not contain any of the special ordering keywords.

- *end*:

  This variable was used to store the scripts that contained the BRACKETEND keyword.

This approach worked well enough and has been extended in the new implementation, now featuring an additional variable called *preamble*, which contains all scripts that contain the PREAMBLE keyword in the correct order. This variable is placed first in the final assembly of the script.

Listing 39: Old: Finding and inserting regular scripts

```
1       elif script != "":
2           regular += f"\nT{tc.tr_test_id} - {tc.tr_case.title}\n"
3           regular += script
```

If the script did not contain any of the special ordering keywords, it arrives at the code shown in listing 39 and is added to the regular variable.

Listing 40: New: Finding and inserting regular scripts

```
1       if formatted.script != '':
2                   regular += f"\nT{tc.tr_test_id} - {tc.tr_case.title}\n"
3                   regular += self._add_tab(formatted.script) + '\n'
```

In the new implementation, the code simply checks if the script variable in the formatted object is not empty. Scripts that do not contain any of the special ordering keywords are stored in this variable, and if it exists, it is added to the regular variable.

Listing 41: New preamble variable and final assembly of the script

```
1       preamble: str = ""
2       for preamble_key in sorted(robot_preamble_dict.keys()):
3           preamble += f"\nT{preamble_test_metadata[preamble_key][0]} -
                {preamble_test_metadata[preamble_key][1]} [PREAMBLE]\n"
4           preamble += self._add_tab(robot_preamble_dict[preamble_key]) + '\n'
5
6       combined_script = preamble + start + regular + end
```

In listing 41, the new preamble variable is created by iterating through the *robot_preamble_dict*, which is a dictionary containing the bodies of all the PREAMBLE keywords, and adding the contents of these keywords to the preamble variable. This dictionary is already ordered by the index of each PREAMBLE keyword, making sure that the preamble variable is assembled correctly. This step happens after the start, regular and end variables have been completely assembled, leading into the final assembly of the script, which is a combination of all four variables.

16. Syntax Highlighting [A]

16.1. Introduction

One part of this project presented in this theis, was the development of Syntax Highlighting. As already described in previous chapters, the Script Assembler works by overlaying a custom syntax on top of the standard Robot Framework language using an EBNF grammar, thus allowing annotations like `@VERSION` or `@BRACKETSTART`.

There is a fundamental problem though. Because standard text-editors and native Robot Framework syntax highlighters do not recognize this new grammar, custom annotations would be displayed as plain text, or worse, marked as syntax errors.

For KEBA, the frontend editor needs a way to visually mark these custom elements, because if they are not able to do so, it can lead to worse developer experience and more errors that go unnoticed until execution. Without visual validation, developers make more syntax mistakes.

For example, writing `@VERISON` instead of `@VERSION` or using an invalid argument type, like passing a plain string where a semantic version number is expected, would not be recognized while typing. These mistakes remain unnoticed. The errors are only found during the backend transpilation phase much later, when an error log is generated. This breaks the entire programming workflow and wastes precious time.

To fix this, custom syntax highlighting should be integrated into the web-based pre-existing code editor. The main objective simply was to get immediate visual feedback and validation. Correct keywords, different types of arguments and errors should all be highlighted in unique colors. This should reduce the time needed for test development, with our Script Assembler annotations/modifiers in use, by a lot. Figure 10 shows how the editor is supposed to handle valid and invalid inputs.

```
1
2  @VERSION BIOS <= 1.0.0
3      Execute on Ketop    Valid
4
5  @VERSION BIOS <= invalid
6      Execute on Ketop    Invalid
7
8  @VERSION BIOS EQUALS 1.0.0
9      Execute on Ketop    Invalid
10
11 @VERISON BIOS <= 1.0.0
12     ^TYPO => no syntax highlighting
13
14 @END
15 ^ internal token => no syntax highlighting
16
```

Figure 10.: Syntax Highlighting in Action
(color scheme was modified for better visibility in print)

16.2. Monaco Editor Setup

The frontend application already used the Monaco Editor to provide basic syntax highlighting for standard Robot Framework scripts with the Monarch library [?, ?]. Monaco is the underlying technology of Visual Studio Code and can be used in frontends through packages. The package that was used was outdated and a few other architectural issues had to be fixed, before the syntax highlighting could be implemented.

16.2.1. Legacy Version Conflicts

Over time the frontend ecosystem of the project has been updated to Angular 15, but the existing editor integration was still based on the widely used wrapper library `ngx-monaco-editor`. Unfortunately, this specific package was no longer being updated. It officially only supported Angular up to version 12 [?]. Previously developers used the npm force flag during installation to make the project compile using this already deprecated package, bypassing the dependency conflicts.

This workaround made it possible for the legacy code to somehow continue working, but it also created major risks for maintainability and the build process. Forcing npm installation is a bad practice since it only covers up package mismatches and other

errors. Because of this, there is a major risk that one might get unpredictable runtime behaviors [?]. Since we do not want that, especially later on in production, the package needed to be replaced.

After some research, the best and most widely used package, fitting exactly our requirements, seemed to be `ngx-monaco-editor-v2`. This version is an actively maintained community fork that is compatible with mordern Angular versions [?].

The migration was handled in several steps to ensure a stable base. The `package.json` had to be updated, then the `app.module.ts` was refactored to match the new library and finally the asset paths in `angular.json` had to be updated.

16.2.2. Custom Theme

A custom Monaco theme was required to map the custom syntax to the correct colors. The requirement for that theme, was for the colors to stand out compared to the standard Robot Framework syntax.

The editor worked fine with its predefined default designs, but as soon as a newly created custom design was applied, a runtime error occurred: `Uncaught TypeError: this.themeData.colors is undefined`. This error completely stopped the editor from rendering any custom syntax highlighting.

Looking in the documentation gave me a hint in the right direction. The engine of Monaco is rather strict, so a `colors` object has to be provided for a theme, even if it is empty and never used [?].

The solution was to add an empty `colors` object to the theme definition. Listing 42 shows the working theme configuration, which includes the required `colors` property alongside the specific token rules.

Listing 42: Custom monaco theme including the required color object

```
// Definition of the custom language theme
(<any>window).monaco.editor.defineTheme('robot-theme', {
  base: 'vs-dark',
  inherit: true,
  rules: [
    { token: 'variables', foreground: '#9CDCFE' },
    { token: 'keywords', foreground: '#ffe18f', fontStyle: 'bold' },
    { token: 'modifier', foreground: '#f09c00', fontStyle: 'bold' },
    { token: 'invalid', foreground: '#ff0000', fontStyle: 'underline italic' },
    // [...]
  ],
  colors: {} // Required to prevent 'this.themeData.colors is undefined'
});
```

16.3. Monarch Lexer for Custom Annotations

After the editor and theme were successfully configured, the core task was to define the syntax rules for the custom grammar of the Script Assembler. The already in use Monaco Editor uses Monarch for syntax highlighting, which works declaratively. Developers define lexers using JSON configuration objects instead of writing complex parser code [**?**].

16.3.1. Tokenization and State Machine

Monarch basically just uses regex to handle the tokenization. The step-by-step-process starts with the lexer, that reads the document text in order and tries to match the defined regular expression patterns. When it finds a match, the matched text is assigned a specific token class. This can be a specific keyword or a comment or it could also just be a newly defined custom modifier. The editor then maps these token classes to the specific color values defined in the active theme.

A global list of regex rules is sufficient for simple languages. But for the custom annotations there is a need for context-aware parsing. For example, the arguments that follow an `@VERSION` annotation should be highlighted, but if the same argument is written in a comment it should not.

Monarch's state machine solved this problem. The lexer starts in a default `@root` state, when a specific pattern is matched, the lexer jumps into a different, explicitly defined state, and different regex rules apply in this new state.

16.3.2. Handling Modifiers and Arguments

The annotations can be roughly divided into two categories. Those without any arguments and those with specific argument structures.

Modifiers without arguments, such as `@BRACKETSTART`, are simple to implement. As shown in Listing 43, when the lexer encounters this keyword, it transitions to an `@endOfLine` state. This state makes sure that no more text is accepted on this line.

Modifiers with arguments are more difficult, because hey require a more complex sequence of state transitions. The modifier `@VERSION` is a good example of this as the version check needs three different components. Also important is, they have to be in an

Listing 43: Monarch tokenizers root state

```
tokenizer: {
  root: [
    // Standard Robot Framework variables
    [/[\$@]\{[\w_/]+\}/, 'variables'],

    //#region MODIFIERS
    [/@(?:PREAMBLE)\b/, 'modifier', '@preambleArg'],
    // Modifiers without any arguments transition to @endOfLine
    [/@(?:BRACKETSTART|BRACKETEND)\b/, 'modifier', '@endOfLine'],
    // Modifiers with complex arguments transition to their specific states
    [/@(?:VERSION)\b/, 'modifier', '@versionArgsIdentifier'],
    [/@(?:OS)\b/, 'modifier', '@osArg'],
    //#endregion

    // [...]
  ],
// ...
```

exact order, starting with an identifier (the BIOS or software name), than a comparison operator (e.g., <, >=), and at last a valid semantic version string.

A chain of states was implemented, Root to identifier, Identifier to operator, Operator to version. That means that when `@VERSION` is found in the `@root` state, the lexer transitions to `versionArgsIdentifier`. Once a valid word is found there, it transitions to `versionArgsOperator` and finally to `versionArgsSemver`.

This "state machine" approach guarantees that the syntax highlighting exactly fits to the strict sequence that the parser needs. Listing 44 shows this state chain in detail.

Listing 44: State transitions for the `@VERSION` modifier

```
versionArgsIdentifier: [
  // Entry point: match a word, then proceed to operator state
  [/[\w_]+/, 'arg.word', '@versionArgsOperator'],
  [/\n/, 'invalid', '@root'],
  [/./, 'invalid']
],
versionArgsOperator: [
  // Match comparison operators, then proceed to semantic version state
  [/(?:[<>]=?)|(?:[!=]=)/, 'arg.operator', '@versionArgsSemver'],
  [/\n/, 'invalid', '@root'],
  [/./, 'invalid']
],
versionArgsSemver: [
  // Match standard semantic versioning format
  // https://semver.org/#is-there-a-suggested-regular-expression-regex-to-check-a-semver-string
  [/(0|[1-9]\d*)\.(0|[1-9]\d*)\.(0|[1-9]\d*)(?:-((?:0|[1-9]\d*|\d*[a-zA-Z-][0-9a-zA-Z-]*)(?:\.(?:0|[1-9]\d*|\d*[a-zA-Z-][0-9a-zA-Z-]*))*))?(?:\+([0-9a-zA-Z-]+(?:\.[0-9a-zA-Z-]+)*))?/, 'arg.version'],
  [/\n/, '', '@root'],
  [/./, 'invalid']
],
```

The regex for the semantic version argument might look complicated and hard to maintain, but it is the officially provided regex by the organization behind semantic versions.

16.4. Troubleshooting and Error Handling

After solving the correct highlighting of valid syntax, the next tasks were to handle edge cases and wrong inputs.

During the implementation of the lexer, two main problems appeared, firstly to prevent "state bleed" over multiple lines, secondly to provide visual error feedback without using a dedicated Language Server.

16.4.1. Fixing State Bleed

The custom EBNF grammar for the Script Assembler is strictly line-bound, so an annotation has to be strictly on a singular line. So if a line break occurs the editor should return to the standard context used for the syntax highlighting of the Robot Framework.

In the beginning, this caused a big conflict with Monarch's stateful tokenization. When the lexer goes into a state like `versionArgsIdentifier`, it just stays inside it until an explicit command such as `@root` or `@pop` is triggered. If a developer typed the keyword `@VERSION` and then pressed the Enter key without any arguments, the lexer got stuck in the `versionArgsIdentifier` state on the next line. Because of this, normal Robot Framework code on the following lines was wrongly checked against the version argument regular expressions. This completely destroyed the syntax highlighting for the whole rest of the file (see Figure 11 for example).

```
1
2  Setup Test Environment
3      Execute On Ketop    echo Syntax Highlighting works
4
5  @VERSION BIOS <= 1.1.1
6      Execute On Ketop    State Bleed
7
```

Figure 11.: State bleed in the editor when line breaks are not included

To fix this "state bleed", the lexer had to recognize line breaks as termination triggers. Normally, Monarch works line-by-line and does not pass line break characters to the regex engine. To change this, the `includeLF` property was enabled in the language configuration (see Listing 45).

Listing 45: `includeLF` in the Monarch config

```
(<any>window).monaco.languages.setMonarchTokensProvider('robot', {
  ignoreCase: true,
  includeLF: true, // Exposes newline to the regex engine
  tokenizer: {
    // tokenizer rules ...
  }
});
```

With `includeLF` set to true, line breaks can be recognized. As can be seen in the state machine definition in Listing 44, each argument state received a new rule for handling the newline character: `[/\n/, 'invalid', '@root']`. Sometimes the lexer just resets to the `@root` state without marking an error. This is used when a line break is allowed and does not count as a syntax mistake in that specific case. This ensures that a line break always returns the lexer to the `@root` state and so protects the highlighting of all following code.

16.4.2. Visualizing Syntax Errors

A major goal for the fronted-editor was to show syntax errors to the user right away, but there was a strict rule for this. The error highlighting had to work only through the Monarch configuration, because a complex language server or an extra linter were not an option, as tools like that just use too many resources.

This was quite a problem because Monarch is really just a lexical analyzer, meaning a lexer, and not a semantic parser [?]; [?, pp. 5-8]. It only uses regular expressions to map strings to token classes. It does not know anything about “errors” or “warnings”. It also cannot draw the typical red wavy lines, that are used for code defects in modern IDEs.

To solve this problem, a visual workaround had to be built with a custom token class called `invalid`. In the custom Monaco Theme (see Listing 42), this category got a unique style with red text-color, italic style and an underline.

This `invalid` category then served as a catch-all mechanism in the state machine. Monarch always checks the rules of a state from top to bottom [?]. By placing the rule `[/./, 'invalid']` at the very end of a states rule list, any character that does not match the expected valid patterns or the newline reset automatically falls through to this fallback rule.

A good example for this is the modifier `@BRACKETSTART`. As it does not allow any arguments, the following happens by running the system. When the lexer sees this exact keyword, it switches to the `endOfLine` state (see Listing 46).

Listing 46: The `endOfLine` state

```
1  endOfLine: [
2    [/\n/, '', '@root'], // Valid: line ends, return to root
3    [/./, 'invalid']     // Error: any other character is invalid
4  ],
```

For example for the case, when a developer accidentally types a `@BRACKETSTART argument`, the lexer first recognizes the annotation, jumps to the `endOfLine` state. There, it expects the end of the line. Instead, it encounters a space, and the word "argument". Because these characters do not match the `/\n/` regex, they fall through to the `/./` rule.

As a result, the string "argument" turns red, gets underlined and italic, and by doing so, shows the developer a syntax error. The user sees a clear syntax error right away without backend validation. Real-time error feedback is a useful feature, and even better, everything remains strictly within the boundaries of the Monarch lexer architecture.

16.5. Limitations

Being able to catch errors early with the syntax highlighting was definitely worth the addition. But even with all the effort, there is a limitation that could not be fixed, since there is a conflict between the rules of the two used languages, which could only be fixed through a custom Language Server.

The EBNF grammar for the backend is strictly case-sensitive. That means, all annotations and modifiers like `@VERSION` or `@OS`, have to be in uppercase, and different spelling leads to a syntax error during transpilation.

The standard Robot Framework language on the other hand works differently. Here upper and lower case do not matter, the language is case-insensitive [?]. Because of this, the `ignoreCase` property of the Monarch lexer must be set to true globally, if not, the normal keywords and variables do not get the correct highlighting (see example in Figure 12).

ignoreCase: true (case-insensitive)

```
1
2  @VERSION only_correct_version < 1.0.0
3      Do Something
4
5  @vErSiOn wrong_but_still_highlighted < 1.0.0
6      Do Something
7
8  ExEcUtE oN kEtOp    echo "technically right"
9  Execute on Ketop    echo "also correct"
10
```

ignoreCase: false (case-sensitive)

```
1
2  @VERSION only_correct_version < 1.0.0
3      Do Something
4
5  @vErSiOn wrong_and_not_highlighted < 1.0.0
6      Do Something
7
8  Execute on KeTop    echo "only one highlighted"
9  ExEcUtE oN kEtOp    echo "technically right"
10 Execute on Ketop    echo "should also be correct"
11
```

Figure 12.: Case sensitivity issues in syntax highlighting

The Monarch API only allows this setting for the entire language, because it is only possible through the global `ignoreCase` property defined in the `IMonarchLanguage` interface. A custom override for one tokenizer rule(`IMonarchLanguageRule`) or a single state is not possible. Therefore, the setting affects everything, the entire tokenizer. In the implemented editor configuration this global setting also affects the custom annotation rules, and as a result, if a developer types `@version` in lowercase, the frontend lexer still matches the rule and applies the corresponding syntax highlighting. But with the same script, the validation crashes when the backend EBNF parser takes the script. The reason is that the parser strictly needs the uppercase syntax [**?**].

This limitation leads to a small difference between the visual representation and the backend, but it is an unavoidable compromise. Disabling `ignoreCase` globally

completely breaks the standard Robot Framework code highlighting, and that code is the vast majority of the file. That is why it is more important, that the base language simply has to function. This was the basis for the final architectural decision together with KEBA, even if it means the developer will only notice the error during transpilation and not while typing.

17. Extensibility [F]

One of the most important requirements given by our project partner is that extending the project, by for example adding new keywords to the parser, should be as easy as possible.

This chapter will go over the necessary steps required to define a new keyword or to write a seperate implementation of the parser.

17.1. Possibilities

The parser is designed to make it particularly easy to define new keywords.

An option or a modifier may be defined to use, require, or optionally allow for an argument. Other keywords or parts of the parser may also be changed to react to the parser's current internal values, to the context of the scope that the keyword is used in, or to new or existing input data.

If needed, it is possible to define keywords to allow for complex logic, for instance by adding an option that sets a variable to a chosen value and adding a modifier to compare it to a different variable.

What follows are a few of examples of keywords that we, for one reason or another, consider to be potentially useful within the context of the TRP project.

17.1.1. @LOADTEMPLATE

In this example, `@LOADTEMPLATE` is an option that takes a *template name* as argument. *Templates* could be passed as a dictionary, where each template name maps to a snippet of Robot Framework code that is commonly re-used across seperate test files.

When this option is encountered, the parser could append the code snippet that belongs to the respective template to the current script or active bracket.

This might allow for the reduction of code duplication, which could increase efficiency and maintainability.

17.1.2. @RETRYAMOUNT

In this example, `@RETRYAMOUNT` is an option that takes an integer as argument.

This argument specifies, how often a test case should be retried in case of error. This could be useful for test cases that are not deterministic, such as ones that include randomizers or fetch an external Uniform Resource Locator (URL).

17.1.3. @BETWEENCASES

In this example, `@BETWEENCASES` is a modifier that takes no argument.

Its block body would get inserted between every test case that is to be run. This could be useful, for instance, logging current timestamps of every test case to a KeTop, or for checking if a signal, that is supposed to terminate the entire execution flow when set, has been received.

17.2. Limitations

Unfortunately, there are also some use cases where the core foundation that the parser is built on may not be sufficient. There are examples of features that, while they might be desirable for certain uses, are however thought to be particularly challenging to implement without rethinking the project from the ground up.

17.2.1. Top-to-bottom processing

Aside from the preprocessor, the parser traverses the file from the top to bottom. This means that, at any point of the file, it is currenty unaware of all keywords that are defined below, as the parser has not reached that point of the file yet.

17.2.2. Naming

The names of keywords must not include spaces and must be preceded by an `@` character. Additionally, modifiers cannot have the same name as options, as they are differentiated by name only.

17.2.3. Multi-line

Since arguments are defined to end when there a line break is encountered, it is not possible to have arguments that span across multiple lines.

Additionally, a keyword is only considered if, aside from inline whitespace, it is at the beginning of a line.

17.3. Adding a Keyword

In general, there are three steps required to add a custom option or modifier to the parser:

1. The keyword should be added to the EBNF, which, from the root of the project, can be found in `./TRP/config/script_assembler_ebnf.txt`.
2. The keyword with its implementation should be added as a case to the parser. From the root of the project, the parser is found in `./TRP/broker/managers/case/script_assembler`, and, within that folder, the core implementation of the keyword in `option_modifier_format.py`.
3. The syntax highlighting for the keyword should be added. The definition for keywords used in syntax highlighting can be found in `Angular/src/app/app.module.ts`.

17.4. Other purposes

Since the parser treats Robot script lines as arbitrary pieces of text, and lines that do not start with an `@` symbol generally pass through unchanged, the Script Assembler project may be used for purposes that transcend the domain of the TRP project that it was initially made for.

In general, any project where simple, custom keywords may be needed to extend the capabilities of any text-based file might be a good use for Script Assembler.

For instance, consider something that has nothing to do with TRP, like a product specification. Critical pieces of information that should not be given away to third parties could, for instance, be defined in a custom modifier named `@SECRET`. Then, a boolean may be added as an input to the parser which decides whether the output should be a version of the specification that intended for internal use, or one that is fit to be sent to a third party.

18. Database [X]

Besides the extension of the Script Assembler's capabilities and the creation of a new EBNF language that makes the software more extendable, another main focus of the project is the creation of a mockup of the TestRail database. This mockup is not only made for testing purposes, but allows the user to work independently of the TestRail instance. The mockup is designed to be as close to the real TestRail database as possible, allowing a seamless switch between the two instances. This means that all functionalities that are available in the real TestRail database are also available in the new mockup.

This section aims to give an overview of the different types of databases, the structure and functionalities of the TestRail database specifically, the resulting design and implementation of the mockup database, and the way the database switch is finally implemented.

18.1. Types of Databases

Depending on the requirements of a given project, different types of databases are suitable for different use cases. It is important to analyze what qualities are needed and which qualities are negligible to find the best fitting solution.

The most important qualities of a database type can be expressed in the following categories:

- **Data Model:**
  The data model of a certain database type describes how the data is structured and stored. This can be in the form of tables, documents, graphs or other structures. The data model has a notable impact on aspects like performance and scalability.
- **Storage Architecture:**
  The storage architecture of a database type not only describes how the data is

stored, but also how it may be accessed, managed and distributed. This can be in the form of a single server, a cluster of servers or even a distributed network of servers across the globe.

- **Query Model:**
  The query model of a database type describes the means of accessing the data. This can occur in the form of a specifically designed query language, an API or other methods. This has an undeniable impact on the ease of use of the database and how the user writes and optimizes queries.

Based on these qualities, various databases types have been developed, each with their own unique advantages and disadvantages. Some of the most common types of databases include but are not limited to relational databases, object-oriented databases and NoSQL databases, which can be further categorized by the data model they use, such as document stores, key-value stores, column-family stores and graph databases [**?**].

18.2. Relational Databases

For the purpose of this project, we will take a closer look at relational databases, as this is the type of database that TestRail uses.

18.2.1. Structure of Relational Databases

Relational databases are based on the relational model, which organizes its data into tables, consisting of rows and columns. Each table represents a specific entity and each row in the table represents an instance of that entity. The columns in the table represent the attributes of the entity.

Relationships between these entities can be established through the use of keys. A primary key is a unique identifier for each instance of an entity, while a foreign key is a reference to the primary key of another entity. Meaningful relationships between entities can be built with the usage of these keys, allowing complex queries to be executed across multiple linked entities [**?**].

Grouping in queries allows for the aggregation of data based on specific attributes, while joins allow for the combination of data from multiple tables based on the established relationships.

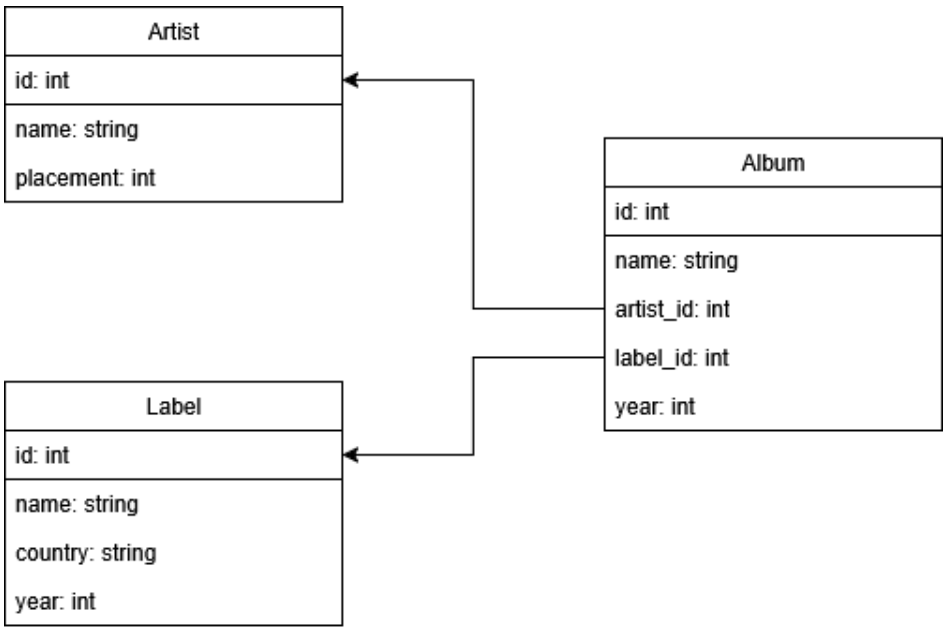


Figure 13.: Simple Diagram showcasing the relationship between three entities in a relational database.

In figure 13, we can see a simplified example of a relational database structure. The three entities *Artist*, *Label* and *Album* are represented as tables, with their respective attributes as columns. The relationships between the entities are established through the usage of primary and foreign keys, allowing for queries to be executed across multiple linked entities. For example, we can query for all albums by a specific artist or a specific label.

18.2.2. ACID

ACID is an acronym that stands for the four key properties of a safe database transaction:

- **Atomicity:**
  Atomicity ensures that a transaction, which is often a sequence of multiple operations, is treated as a single unit. This means that either all operations within a given transaction are executed successfully, or none of them are. If any operation of the transaction fails due to an error, a power outage or any other issue, the entire transaction is rolled back to its previous state before the transaction began. This guarantees that the database remains consistent and prevents data corruption or loss that could occur if only part of the transaction were to be executed.

- **Consistency:**
  Consistency ensures that the database can only be brought from one valid consistent state to another, while following all defined rules, constraints and triggers. Invalid transactions that violate any of the defined rules are rolled back accordingly, ensuring that the database remains consistent.

- **Isolation:**
  Isolation ensures that concurrent transactions have no impact on each other. This means that the effects of a transaction in progress are not visible to other transactions until it completes successfully. This prevents a number of issues that would otherwise arise from concurrent transactions, like dirty reads, where one transaction reads data that has been modified by another transaction but not yet fully committed, potentially leading to incorrect results.

- **Durability:**
  Durability ensures that once a transaction has been successfully completed and committed, the state of the database persists even in the case of a crash, power failure or any other issue.

Databases that adhere to these properties are considered to be reliable and safe for use in critical environments, as they guarantee that the data not only remains consistent, but also that it is not lost or corrupted due to unforseen events [**?**].

18.2.3. Advantages and Disadvantages of Relational Databases

One of the main strengths of relational databases is their ability to handle long and complex queries across multiple linked entities with the usage of foreign keys and joins. The consistency and integrity of the data is guaranteed by the ACID properties. On top of that, relational databases are some of the most widely used and well-known types of databases, which means that there are a lot of resources and a large community available for support and development.

However, relational databases can struggle with scalability and performance when dealing with large volumes of data or high traffic. The fixed schema of relational databases can also be a disadvantage, as it can make it difficult to adapt to changing requirements or to handle unstructured data. Additionally, the execution of complex queries can lead to performance issues on larger datasets.

18.3. The TestRail Database

TestRail is a web-based test management tool that provides a wide range of features for managing and organizing test cases, test runs and test results. The relational database in the background of TestRail is used to store all of the data related to these features. The user can interact with the database directly by using the TestRail API, although the database itself is not directly accessible to the user.

Thus, for the creation of the mockup database, the structure and functionalities of the TestRail database have to be studied and analyzed based on the API documentation and the structure of its response data. While the mockup database doesn't have to be an exact replica of the TestRail database, the functionalities used by the Script Assembler need to deliver responses with identical structure to ensure the switch between databases can occur without any issues.

In order to achieve this, the most important entities and their relationships have to be identified.

18.3.1. Test Cases

Test cases can be seen as the most fundamental entity in the TestRail system, as they represent the individual tests that are to be executed. Test cases have a number of attributes, such as a title, description, type and the test steps that need to be executed. Test cases are furthermore linked to test suites and sections.

18.3.2. Projects, Suites and Sections

Projects are the highest level of organization in TestRail, representing a specific testing effort or initiative. Each project can contain multiple test suites, which are collections of test cases. The test cases of Suites can further be organized into sections, allowing for more specific grouping and categorization. These relationships are an essential part of the database structure that needs to be replicated.

18.3.3. Test Runs

A test run represents a specific execution of test cases. Test runs are linked to test cases and contain information about the execution, such as the status of each case, the tester who executed the run and any relevant comments or other metadata. Test runs make the tracking and management of test executions possible. Importantly, a test run can only be created for test cases that are contained in one singular test suite.

18.3.4. Test Plans

Test plans are used to organize and manage test runs. TestRail allows the user to group multiple test runs into a test plan, which alleviates the need to manage each run individually.

18.3.5. Test Results

Test results represent the outcome of a test case within a test run. Test results mainly consist of a status, which can contain, but is not limited to:

- **Passed**
  A test case is marked as passed when it has been successfully executed and all results are as expected.
- **Failed**
  A test case is marked as failed when it has been executed but the results are not as expected, meaning that the tested functionality does not work as intended.
- **Untested**
  A test case that hasn't been assigned a status yet is marked as untested.
- **Retest**
  A test case is marked as retest when it has failed previously, but has been fixed and is now ready to be tested again.
- **Blocked**
  A test case is marked as blocked when it cannot be executed due to an error that is not related to the functionality being tested.

18.3.6. Users

Users represent the individuals who interact with the TestRail system. Users can have different roles and permissions within the system. They can be assigned to test runs, test plans or other entities as creators. With the user entity it is possible to track who is responsible for which run and plan.

18.4. PostgreSQL

For the implementation of the mockup database, PostgreSQL was chosen as the database management system, as it is an open-source database that is not only widely used but also well-known for its reliability and performance.

PostgreSQL is not only a database management system that the team is familiar with, but it also provides all the necessary features and capabilites needed for the implementation of this part of the project. It supports the relational data model, which is needed for replacing the TestRail database, and it guarantees the reliability and consistency of its data with the ACID properties.

18.4.1. pgAdmin

The open-source management tool *pgAdmin* is used for the administration of the PostgreSQL database, as it provides an easily navigable user interface for management, query execution and other administrative tasks.

The current version of pgAdmin, which is pgAdmin 4, provides a wide range of features for managing and administering PostgreSQL databases, some of which include but are not limited to the following: [**?**].

- **SQL query tool:**
  Inside pgAdmin, there is a built-in SQL editor that allows users to write and execute SQL queries directly against the database. This tool provides syntax highlighting, further aiding the user in writing correct queries.
- **Direct data editing:**
  The tables in a database can be directly edited through the interface, allowing for quick and easy modifications of single points of data without the need to write and

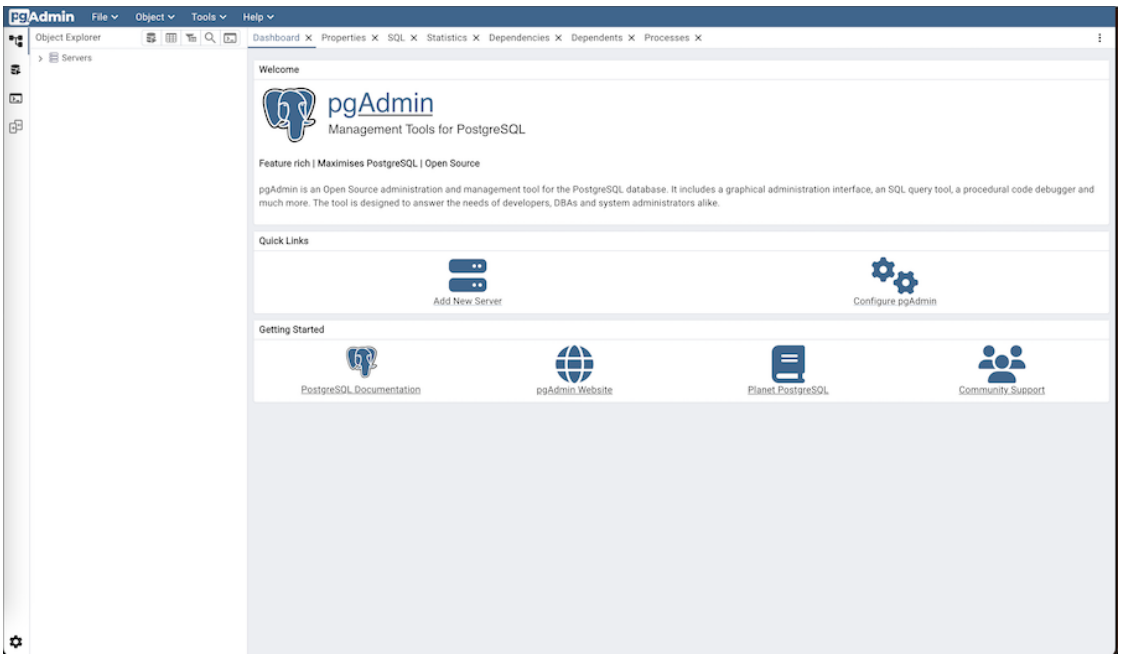


Figure 14.: Screenshot of the pgAdmin interface [**?**].

execute SQL queries. Caution should be taken when using this feature however, as the quick and easy nature of this feature can lead to unintended consequences.

- **Management of multiple databases:**
  The pgAdmin interface allows the management of multiple databases within the same interface, allowing for intuitive switching between independent databases.

18.5. First Approaches

Early into the development of the database, two main approaches were considered for how the database should interact with the backend of the main system.

One approach was to create an API that would allow the main system to interact with the database through API calls. This would have resulted in a clear separation between the database and the main system.

The other approach was to work in the preexisting backend to connect to the database directly. This approach was ultimately chosen, as it not only makes the switch between the two databases simpler, but also integrates the functionality of the new database directly into the main system. This approach was further endorsed by the supervisor of the project, as it helps the system to eventually transition away from the TestRail database and towards the new mockup database, which allows for more independence and expandability in the future.

The python library *psycopg* is used for the connection between the main system and the PostgreSQL database. This library provides a simple and efficient way to connect to the PostgreSQL database and execute various commands and queries. It also provides support for connection pooling, which can help to improve the performance of the database interactions.

18.6. Database Connections in the System

The following section aims to give a brief look into the structure and functionalities of not only the existing TestRail database connection, but also the new PostgreSQL mockup database. The goal is to show how the new database is designed to replicate the implementations of the TestRail connection, while also highlighting clear differences on a technical level.

18.6.1. Existing TestRail Functionality

The existing funtionalities concerning the TestRail connection is contained within the TRSession class, which is responsible for all interactions with the database. This class contains functions for retrieving and creating test cases, runs, plans and results, as well as other relevant data.

The communication with the TestRail database is achieved through the usage of the TestRail API, which provides all necessary reading and writing functionalities.

Listing 47: Code snippet of the get_case_title function within the TRSession class.

```
1
2       def get_case_title(self, case_id: int, safe_name: bool = False) -> str | None:
3           response = self.do_request(f"get_case/{case_id}", TRConst.REQUEST_TYPE_GET)
4           if not response:
5               return None
6           if len(response) <= 0:
7               self.log.warning(f"could not find a result for test case: {case_id}")
8               return None
9           if not safe_name:
10              return response['title']
11          return response['title'].replace(":", "")
```

In listing 47, one of the functions of the TRSession class can be seen. This function is responsible for retrieving the title of a test case based on its ID. Using the ID of the wanted test case, a GET request is sent to the TestRail API, which then returns the relevant data. The function then processes the response data and returns the title.

All functions of the TRSession class follow a similar structure, with the type of interaction with the API being the main difference. For example, functions that create new entities in the database use POST requests and send the relevant data in the body of the request.

18.6.2. DBSession

The DBSession class is designed to replicate all used functionalities of the exisitng TRSession class, while replacing the communication with the TestRail API with a direct connection to the PostgreSQL database, using the *psycopg* python library. It is strictly necessary that the functions of the DBSession have the same name, parameters and return values as the functions of the TRSession class, in order to avoid major conflicts when switching between the two database connections.

Listing 48: Code snippet of the get_case_title function within the DBSession class.

```
1
2       def get_case_title(self, case_id: int, safe_name: bool = False) -> str | None:
3
4           if case_id == None:
5               print("Did not get a case_id")
6               return None
7
8           if case_id < 0:
9               print(f"Invalid value for case_id: {case_id}")
10              return None
11
12          self.cursor.execute("SELECT title FROM test_cases WHERE id = %s",
                (case_id,))
13
14          result = self.cursor.fetchone()
15
16          if result != None:
17              return str(result[0])
18
19          return None
```

In listing 48, the same function as in listing 47 can be seen, but this time within the DBSession class. Thus, while the parameters and return value of the function are identical to the one in the TRSession class, the implementation is different.

Instead of interacting with an API, this function uses a connection to the PostgreSQL database to execute a SQL query that retrieves the title of the specified test case.

18.7. Integration of the DBSession Class

Since the DBSession class replicates all essential functions of the TRSession class, the switch between database usages can be achieved easily.

Any occurrence of the TRSession class within the backend of the system is now denoted as a TRSession or DBSession, using a pipe symbol. This means that at system startup, the appropriate session class has to be identified and initialized accordingly.

Listing 49: Code snippet showcasing a function that takes a parameter that can be either a TRSession or a DBSession.

```
1
2       def fetch_cases(self, testrail: TRSession | DBSession):
3           # Function implementation
```

This way, the TRSession functionalities remain unaltered, while the possibility of using the DBSession is a new feature that can be easily switched on or off.

18.7.1. Switching Between Databases

The switch between the two databases is implemented in a way that allows the user to easily and definitively choose which database should be used at the execution of the python script. This is achieved by using a command line argument that can be passed when executing the script. The argument is then processed and the appropriate database instance is initialized based on if the argument is present or not.

```
python ./broker.py
```

Here, the standard TestRail database is used, as no argument is passed to the script.

```
python ./broker.py -db
# OR
python ./broker.py --database
```

Here, the PostgreSQL mockup database is used, as the argument is passed to the script. The argument can either be a short version, or a long version, as both are processed in the same way and lead to the same result.

18.8. The PostgreSQL Database

Since the structure of the TestRail database is not publicly available, the structure of the PostgreSQL mockup database is based on the structure of the response data the

TestRail API provides. All entities contain their respective attributes, as well as keys that establish the relationships between the different entities.

It is important to note that the relationships between entities are not made with the usage of foreign keys, as the logic for the relationships is implemented in the backend of the system, and the database is only used for storing data. While this is not the most efficient way to implement the relationships between entities, as a mockup database, it is sufficient for the purpose of this project. Despite that, in section 18.9, potential future steps are discussed, one of which is the implementation of proper relationships between entities with the usage of foreign keys.

The database contains tables for all the entities of the TestRail system, that are relevant for the functionalities of the Script Assembler. All entities are listed below, along with their respective attributes.

Every entity contains an *id* attribute, signifying the unique identifies for each instance of the entity, with the datatype of *serial*, which means that the integer value of the attribute is automatically incremented with each new instance of the entity. Since this attribute is present in all entities, it will not be listed for each entity separately.

18.8.1. Case Types

Case Types are used to categorize the test cases based on their type, such as functional, performance or security test cases. This is mainly used to group test cases together and strictly serve the organizational purposes of the system.

This table has the following attributes: [**?**]

- **is_default:**
  This attribute is a boolean value that is set to true for one specific case type, which is assigned as the default case type. All other case types have this attribute set to false.
- **case_name:**
  This attribute is a string value that contains the name of the case type.

18.8.2. Groups

Groups are used to categorize users based on their roles, permissions or tasks. Similarily to case types, groups are mainly used for organizational purposes.

This table has the following attributes: [**?**]

- **name:**
  This attribute is a string value that contains the name of the group.
- **user_ids:**
  This attribute is an array of integers that contains the IDs of all users assigned to the specific group.

18.8.3. Priorities

Priorities are used to order test cases in the system. Test cases with a higher value of priority are considered more important and are therefore executed before others with a lower value of priority. This also influences the ordering of test cases in the user interface, as well as the order they are assembled in the final script.

This table has the following attributes: [**?**]

- **is_default:**
  This attribute is a boolean value that is set to true for one specific priority, which is assigned as the default priority. All other priorities have this attribute set to false.
- **prio_name:**
  This attribute is a string value that contains the name of the priority.
- **priority:**
  This attribute is an integer value that contains the priority level, with higher values corresponding to a higher priority.
- **short_name:**
  This attribute is a string value that contains a short name alternative for the priority.

18.8.4. Projects

Projects are the uppermost level of organization in the system. They contain other entities such as test suites and by extension sections and test cases. Projects represent a specific testing effort or testing environment for an entire system.

This table has the following attributes: [**?**]

- **announcement:**
  This attribute is a string value that contains the announcement message for the project, which provides more information about the project and its purpose.
- **default_role_id:**
  This attribute is an integer value pointing to the ID of the role that is assigned as the default role for the project. This role is automatically assigned to all users that are added to the project, unless they are assigned a different role.
- **default_role:**
  This attribute is a string value that contains the name of the default role.
- **is_completed:**
  This attribute is a boolean value that indicates whether the project has been completed or not.
- **project_name:**
  This attribute is a string value that contains the name of the project.
- **show_announcement:**
  This attribute is a boolean value that indicates whether the announcement message of the project should be shown to users or not.
- **url:**
  This attribute is a string value that contains the URL of the project.
- **users:**
  This attribute is an array of integers that contains the IDs of all users assigned to the project.
- **completed_on:**
  This attribute is an integer value that contains the UNIX timestamp of when the project was completed.

18.8.5. Test Suites

Test Suites are collections of test cases that are grouped together based on their purpose or functionality.

This table has the following attributes: [**?**]

- **description:**
  This attribute is a string value that contains the description of the specific test suite.
- **name:**
  This attribute is a string value that contains the name of the test suite.
- **project_id:**
  This attribute is an integer value that contains the ID of the project that the test suite is assigned to.
- **url:**
  This attribute is a string value that contains the URL of the test suite.

18.8.6. Sections

Sections further divide and organize the test cases within a test suite.

This table has the following attributes: [**?**]

- **depth:**
  The depth attribute is an integer value that indicates the level of the section within the hierarchy of sections. The lower the value, the higher the section is in the hierarchy.
- **description:**
  This attribute is a string value that contains the description of the specific section.
- **display_name:**
  This attribute is a string value that contains a name of the section as it should be displayed in the user interface.
- **name:**
  This attribute is a string value that contains the name of the section.

- **parent_id:**
  Sections can be nested within other sections, which can be achieved by assigning a section as the parent of another. This attribute is an integer value that contains the ID of the parent section, if there is one. If this section is in the uppermost level of the hierarchy, it does not have a parent and therefore this attribute is set to null.
- **suite_id:**
  This attribute is an integer value that contains the ID of the test suite that the section is assigned to.

18.8.7. Test Cases

Test cases represent the individual tests that can be executed. Considering that the TestRail API provides a lot of data related to test cases, the test case entity contains a large number of attributes. Many of these are not used but are included regardless to ensure the structure remains true to the TestRail database, to avoid any issues when switching between the two database types.

The most important attributes of the table include the following: [**?**]

- **title:**
  This attribute is a string value containing the title of the test case.
- **section_id:**
  This attribute is an integer value containing the ID of the section that the test case is assigned to.
- **suite_id:**
  This attribute is an integer value containing the ID of the test suite that the test case is assigned to.
- **type_id:**
  This attribute is an integer value containing the ID of the assigned case type.
- **priority_id:**
  This attribute is an integer value containing the ID of the assigned priority.

- **created_by:**
  This attribute is an integer value containing the ID of the user that originally created the test case.
- **created_on:**
  This attribute is an integer value containing the UNIX timestamp of when the test case was originally created.
- **updated_by:**
  This attribute is an integer value containing the ID of the user that last updated the test case.
- **updated_on:**
  This attribute is an integer value containing the UNIX timestamp of when the test case was last updated.
- **custom_automation_instructions:**
  This attribute is a string value containing the entire test script pertaining to the test case. This is a custom attribute that is not present in the TestRail database, but is essential for the functionalities of the Script Assembler.

18.8.8. Test Results

Test results represent the outcome of a test case within a test run.

This table has the following attributes: [**?**]

- **assignedto_id:**
  This attribute is an integer value containing the ID of the user that is assigned to and responsible for the test result.
- **comment:**
  This attribute is a string value containing any relevant comments or error messages related to the test result.
- **created_by:**
  This attribute is an integer value containing the ID of the user that originally created the test result.

- **created_on:**
  This attribute is an integer value containing the UNIX timestamp of when the test result was originally created.
- **defects:**
  This attribute is a string value containing a list of defects regarding the test result, if present.
- **elapsed:**
  This attribute is a string value containing the time it took to execute the test. This time is given in the format *"Xm Ys"*, where X is the number of minutes and Y is the number of seconds.
- **status_id:**
  This attribute is an integer value containing the ID of the status of the test result.
- **test_id:**
  This attribute is an integer value containing the ID of the test that the result is assigned to.
- **version:**
  This attribute is a string value containing the version of the system that was tested.

18.8.9. Test Runs

Test runs represent a specific execution of a batch of test cases. Similarly to test cases, test runs have a large number of attributes, many of which are not used but are included regardless to ensure the structure remains true to the TestRail database.

The most important attributes of the table include the following: [**?**]

- **name:**
  This attribute is a string value containing the name of the test run.
- **description:**
  This attribute is a string value containing the description of the test run.
- **assignedto_id:**
  This attribute is an integer value containing the ID of the user that is assigned to the test run.

- **is_completed:**
  This attribute is a boolean value indicating whether the test run has been successfully completed or not.
- **completed_on:**
  This attribute is an integer value containing the UNIX timestamp of when the test run was completed.
- **passed/blocked/untested/retest/failed_count:**
  These attributes are integer values containing the count of test cases within the test run that have the status of passed, blocked, untested, retest and failed respectively.
- **project_id:**
  This attribute is an integer value containing the ID of the project that the test run is assigned to.
- **created_by:**
  This attribute is an integer value containing the ID of the user that originally created the test run.
- **created_on:**
  This attribute is an integer value containing the UNIX timestamp of when the test run was originally created.
- **url:**
  This attribute is a string value containing the URL of the test run.
- **case_ids:**
  This attribute is an array of integers containing the IDs of all test cases that are a part of the test run.

18.8.10. Tests

Tests are executed instances of test cases within a test run. There can be multiple tests for the same test case, as a test case can be executed multiple times through multiple test runs.

This table has the following attributes: [**?**]

- **assignedto_id:**
  This attribute is an integer value containing the ID of the user that is assigned to and responsible for the test.
- **case_id:**
  This attribute is an integer value containing the ID of the test case that the test is an instance of.
- **priority_id:**
  This attribute is an integer value containing the ID of the assigned priority for the test.
- **run_id:**
  This attribute is an integer value containing the ID of the test run that the test is a part of.
- **status_id:**
  This attribute is an integer value containing the ID of the status of the test.
- **title:**
  This attribute is a string value containing the title of the test. This usually refers back to the title of the test case that the test is an instance of, but has a unique name to differentiate it from other instances of the same test case.
- **type_id:**
  This attribute is an integer value containing the ID of the assigned case type for the test.

18.8.11. Users

Users are the individuals that interact with the system. They can have different roles and permissions within the system and can be assigned to various entities within the system, such as test runs or test plans as creators and updaters.

This table has the following attributes: [**?**]

- **email:**
  This attribute is a string value that contains the email address of the user.

- **email_notifications:**
  This attribute is a boolean value that indicates whether the user has email notifications enabled or not.
- **is_active:**
  This attribute is a boolean value that indicates whether the user is active or not.
- **is_admin:**
  This attribute is a boolean value that indicates whether the user has admin privileges or not.
- **name:**
  This attribute is a string value that contains the name of the user.
- **role_id:**
  This attribute is an integer value that contains the ID of the role assigned to the user.
- **role:**
  This attribute is a string value that contains the name of the role assigned to the user.
- **group_ids:**
  This attribute is an array of integers that contains the IDs of all groups that the user a part of.
- **assigned_projects:**
  This attribute is an array of integers that contains the IDs of all projects that the user is assigned to.
- **password:**
  This attribute is a string value that contains the password of the user. This is stored in plain text in the database, which is not ideal for security reasons, but is sufficient for the purpose of the local mock database. In section 18.9, potential future steps are discussed, one of which is the implementation of a system for encrypting or hashing the credentials of all users.

18.9. Potential Future Steps

18.9.1. Easier Creation of Test Cases

TestRail provides a functionality for the creation of test cases through the API, as well as through their user interface. This makes the creation of new test cases in the database an easy task, especially with the user interface, as the user can efficiently create new test cases and other entities without having to worry about the structure of the database itself.

Currently, the creation of test cases in the PostgreSQL database is done manually by either executing SQL queries manually or by making use of the pgAdmin user interface. This is not only time-consuming, but also prone to errors, as the user has to ensure that all necessary attributes of the test case are filled in correctly and that the relationships between entities are established properly.

A potential future step to fix this issue would be to create a user interface that allows for the easier creation of test cases and other entities in the database. This system could either be a separate application that interacts with the database on its own, or it could be integrated into the existing system.

18.9.2. Information Security

For convenience and ease of use, the users of the PostgreSQL database currently have their credentials stored in plain text within their respective columns in the database. While this is not an issue for the mockup database, as it is only used for testing purposes, if the PostgreSQL database approach is to be used in the future as a replacement for the TestRail database, it would be necessary to implement a system for encrypting or hashing the credentials of all users, in order to ensure the security of the database and the data it contains.

This could be achieved by using a hashing algorithm to hash the passwords before storing them in the database, and then comparing the hashed password with the stored hash when a user attempts to log in. This would ensure that actual passwords cannot be retrieved from the database, even if the database is compromised.

Hasing is an irreversible process, meaning that once a password is hashed, it cannot be converted back to its plain text form. Encryption, on the other hand, is a reversible

process, so it would be possible to retrieve the original password from the encrypted form if the key is known. While hashing is more secure for storing passwords, both methods are viable options for ensuring the security of credentials in the database.

18.9.3. Implementation of Proper Relationships

The relationships between entities in the PostgreSQL database are not implemented with the usage of foreign keys, but rather with the usage of keys that are processed in the backend of the system. The logic for the relationships is implemented in the backend, which means that the database is only used for strictly storing data.

This is not the most efficient way to implement relationships, as it can lead to issues with data integrity, as well as performance issues when executing queries that involve multiple linked tables.

The logical next step to improve the implementation of the database would be to implement proper relationships between entities with the correct usage of foreign keys.

Glossary

ACID Atomicity, Consistency, Isolation, and Durability

API Application Programming Interface

ASCII American Standard Code for Information Interchange

EBNF Extended Backus–Naur form

IDE Integrated Development Environment

regex Regular expression

TRP TestRail Procedure

List of Figures

List of Listings

Appendix

Project Timeline

The Script Assembler project was carried out as part of an internship between Filip Schauer, Axel Csomány, Adam Höllerl, and our project partner, KEBA Group AG in Linz, Austria.

In the first week of our internship, we spent most of our time familiarizing ourselves within the TRP environment, discussing potential solutions and developing our EBNF. Afterwards, we made the parser including all of its components, created the mockup database, and made sure that our project was well tested and integrated into the existing system.

Most of the requirements were given by our project partner, Martin Wieser, and we not only did our best to adhere to these requirements, but also gave our own input to allow us to create the best possible solution for KEBA's unique situation.

Our work is in active use within KEBA's professional contexts, and we hope that our project is appreciated by anyone who might find themselves writing test scripts for KeTop devices.

Filip Schauer's Takeaways

While I already had experiences with Python outside of professional environments, this internship was the first opportunity I had gotten to work in a large and well established Python project.

I enjoyed the algorithmic challenge that the parser brought with itself. I believe that it was an important part of my learning process to be able to discuss ideas with a project partner, and to make educated decisions on specific approaches and tools.

Axel Csomány's Takeaways

The learnings and experiences I've gained during this internship were invaluable. My first experience diving into a functioning system and applying major modifications to it was daunting at first, but rewarding by the end. I have learned how to analyze big problems and recognize how to split it up into smaller, more manageable tasks. But most importantly of all, when the situation seems too complicated, take a break and come back later.

Adam Höllerl's Takeaways